

University Of Peradeniya
Faculty Of Engineering
Department of Computer Engineering
CO2050 – Database Systems

**Design and Development of a Complete Database System for
a Forensic Medical Department**

Group – 23

Submission Date - 7/23/2026

Mr.Biswajith Dissanayake

K.A.D.Geethma Chandooa – E/22/180

S.A.T.Inuwara - E/22/154

R.M.Y Hansira – E/22/132

S.L.Y Prabudhika – E/22/288

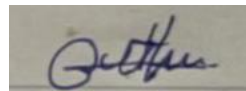
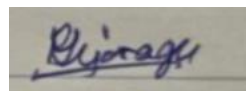
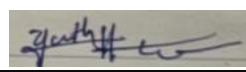
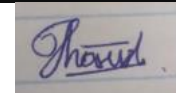
Declaration

I hereby declare that this report entitled "**Design and Development of a Complete Database System for a Forensic Medical Management System**" is the result of my/our own work carried out in partial fulfillment of the requirements for the **CO2040 – Database Systems** course. The work presented in this report has not been submitted, either in whole or in part, to any other university or institution for the award of any degree, diploma, or other qualification.

All sources of information, published materials, and contributions of others used in this report have been appropriately acknowledged and referenced.

I/We affirm that this project was completed with honesty and integrity in accordance with the academic regulations of the institution.

Group No : 23

No.	Name of the Student	Index Number	E-mail address	Signature
1	K.A.D. Geethma Chandoopu	E/22/180	e22180@end.pdn.ac.lk	
2	R.M.Y. Hansira	E/22/132	e22132@end.pdn.ac.lk	
3	S.L.Y. Prabudhika	E/22/288	e22288@end.pdn.ac.lk	
4	S.A.T. Inuwara	E/22/154	e22154@end.pdn.ac.lk	

Acknowledgement

First and foremost, we would like to express our sincere gratitude to our course instructor for providing us with the opportunity to undertake this project and for the continuous guidance, valuable suggestions, and constructive feedback throughout the development of the **Forensic Medical Management System**.

We are also grateful to the Department of Computer Engineering for providing the necessary facilities, learning resources, and academic environment that enabled us to successfully complete this project.

Our heartfelt appreciation goes to all the lecturers and staff members who shared their knowledge and provided support during the course. Their encouragement and expertise greatly contributed to the successful completion of this work.

We would also like to thank our teammates for their cooperation, dedication, and teamwork throughout every stage of the project, from requirement analysis and database design to implementation, testing, and documentation.

Finally, we extend our sincere thanks to our families and friends for their constant encouragement, patience, and moral support throughout the completion of this project.

Abstract

Forensic medical departments are responsible for conducting medico-legal examinations, maintaining patient and case records, managing forensic evidence, and preparing reports required for legal proceedings. Many institutions still depend on manual or fragmented record management systems, which can lead to data redundancy, inefficient case handling, difficulties in retrieving historical records, delayed report generation, and challenges in maintaining the confidentiality and integrity of sensitive medico-legal information. These limitations reduce operational efficiency and increase the possibility of errors in forensic investigations.

This project aims to design and develop a comprehensive **Forensic Medical Database Management System** that provides a centralized and secure platform for managing all major activities within a forensic medical department. The proposed system supports the management of patients, forensic cases, police requests, medical examinations, postmortem reports, medico-legal reports, evidence records, doctors, and departmental information while ensuring data consistency and integrity.

The database was designed using the **Entity-Relationship (ER) model**, followed by relational schema design and normalization up to the **Third Normal Form (3NF)**. Primary keys, foreign keys, and integrity constraints were incorporated to establish relationships among entities and eliminate data redundancy. The system was implemented using **MySQL** with **SQL** for database creation, querying, and data manipulation.

The completed database consists of interconnected entities that accurately represent the forensic medical workflow, enabling efficient case registration, evidence tracking, examination recording, report generation, and secure data retrieval. The proposed system improves the accuracy, reliability, and efficiency of forensic record management while providing a scalable database solution that supports medical professionals, investigators, and judicial authorities.

Keywords: Forensic Medicine, Database Management System, Entity-Relationship Diagram (ERD), SQL, Evidence Management, MySQL

Table of Content

Chapter	Title	Page No.
	Declaration	i
	Acknowledgement	ii
	Abstract	iii
	Table of Contents	iv
	List of Figures	v
	List of Tables	vi
	List of Abbreviations	vii
Chapter 1	Introduction	1
1.1	Background	1
1.2	Problem Statement	2
1.3	Objectives	3
1.4	Scope of the Project	4
1.5	Significance of the Study	5
1.6	Technologies Used	6
Chapter 2	Requirement Analysis	8
2.1	Existing System	8
2.2	Problems in Existing System	9
2.3	Proposed System	10
2.4	Stakeholders	11
2.5	Functional Requirements	12
2.6	Non-functional Requirements	14
2.7	Use Cases	15
2.8	Workflow / Business Process	18
Chapter 3	Database Design	20
3.1	Entity Identification	20
3.2	ER Diagram	21
3.3	Enhanced ER Diagram	22
3.4	Relational Schema	23
3.5	Data Dictionary	29

3.6	Primary Keys	39
3.7	Foreign Keys	40
3.8	Integrity Constraints	41
3.9	Normalization	43
Chapter 4	Database Implementation	46
4.1	DBMS Used	46
4.2	Database Creation	47
4.3	Table Creation	48
4.4	Constraints	52
4.5	Indexes	54
4.6	Views	55
4.7	Stored Procedures	56
4.8	Triggers	57
4.9	Sample Data	58
Chapter 5	User Interface	61
5.1	System Architecture	61
5.2	Login Module	62
5.3	Dashboard	63
5.4	Patient Management	64
5.5	Case Management	65
5.6	Examination & Postmortem Module	67
5.7	Evidence Management	68
5.8	Report Management	69
5.9	Doctor Management	70
5.10	User Authentication	71
Chapter 6	Testing	73
6.1	Test Plan	73
6.2	Test Cases	74
6.3	Validation Testing	76
6.4	SQL Query Testing	77
6.5	Backup and Recovery	82
Chapter 7	Reports	84
7.1	Daily Case Report	84
7.2	Monthly Report	85
7.3	Pending Cases Report	86
7.4	Court Report	87

7.5	Statistical Report	88
7.6	Report Summary	89
Chapter 8	Security	90
8.1	Introduction	90
8.2	Password Security	91
8.3	User Roles	92
8.4	Access Control	93
8.5	Confidentiality	94
8.6	Audit Logs	95
8.7	Database Backup	96
8.8	Summary	98
Chapter 9	Challenges and Future Improvements	99
9.1	Introduction	99
9.2	Challenges Faced During Development	100
9.3	Future Improvements	103
9.4	Summary	107
Chapter 10	Conclusion	108
10.1	Introduction	108
10.2	Objectives Achieved	109
10.3	Benefits of the System	110
10.4	Lessons Learned	112
10.5	Overall Conclusion	113
10.6	Final Remarks	114
	References	116
	Appendices	118

List of Figures

Figure No.	Description	Page No.
Figure 2.1	Workflow / Business Process Flowchart	18
Figure 3.1	Entity Relationship (ER) Diagram	21
Figure 3.2	Enhanced ER Diagram	22
Figure 3.3	Relational Schema	23
Figure 4.1	Database Creation in MySQL Workbench	47
Figure 4.2	CREATE TABLE Statements	48
Figure 4.3	Primary Key Constraint	52
Figure 4.4	Foreign Key Constraint	52
Figure 4.5	UNIQUE Constraint	53
Figure 4.6	CHECK Constraint	53
Figure 4.7	DEFAULT Constraint	54
Figure 4.8	Sample Data in Tables	58
Figure 5.1	System Architecture	61
Figure 5.2	Login Interface	62
Figure 5.3	Dashboard	63
Figure 5.4	Patient Management Interface	64
Figure 5.5	Case Management Interface	65
Figure 5.6	Examination Module	67
Figure 5.7	Postmortem Module	67
Figure 5.8	Evidence Management Module	68
Figure 5.9	Report Generation Module	69
Figure 5.10	Doctor Management Module	70
Figure 6.1	Test Case Results	74
Figure 6.2	Validation Testing Results	76
Figure 6.3	JOIN Query Result	78
Figure 6.4	GROUP BY Query Result	79
Figure 6.5	Aggregate Function Result	80
Figure 6.6	Subquery Result	81
Figure 6.7	View Query Result	82
Figure 7.1	Daily Case Report	84
Figure 7.2	Monthly Report	85
Figure 7.3	Pending Cases Report	86
Figure 7.4	Court Report	87
Figure 7.5	Statistical Report Dashboard	88

Figure 8.1	Firebase Authentication Login Page	91
Figure 8.2	User Role Management	92
Figure 8.3	Access Denied Page	93
Figure 8.4	Protected Patient Records	94
Figure 8.5	Audit Log	95
Figure 8.6	Database Export	96
Figure 8.7	Database Import	97

List of Tables

Table No.	Description	Page No.
Table 2.1	Stakeholders	11
Table 2.2	Technologies Used	6
Table 3.1	Case Table	23
Table 3.2	Examinee Table	24
Table 3.3	Investigation Table	24
Table 3.4	Court Information Table	24
Table 3.5	Police Information Table	24
Table 3.6	Deceased Table	24
Table 3.7	Report Table	25
Table 3.8	Injury Table	25
Table 3.9	Clinical Findings Table	25
Table 3.10	Referral Table	25
Table 3.11	Document Table	25
Table 3.12	Exhibit Table	26
Table 3.13	User Table	26
Table 3.14	Postmortem Findings Table	26
Table 3.15	Hospital Table	27
Table 3.16	Consent Table	27
Table 3.17	Next of Kin Table	27
Table 3.18	Ward Table	27
Table 3.19	Audit Log Table	27
Table 3.20	Doctor Table	27
Table 3.21	Body Identification Table	28
Table 3.22	Histopathology Report Table	28
Table 3.23	Certificate of Receipt Table	28
Table 3.24	Toxicology Report Table	28
Table 3.25	Specimens Table	28
Table 3.26	Data Dictionary	29
Table 3.27	Primary Keys	39

Table 3.28	Foreign Keys	40
Table 4.1	SQL Constraints	52
Table 4.2	Database Indexes	54
Table 6.1	Test Cases	74

List of Abbreviations

Abbreviation	Meaning
DBMS	Database Management System
ERD	Entity Relationship Diagram
SQL	Structured Query Language
PK	Primary Key
FK	Foreign Key
UML	Unified Modeling Language

Abbreviation	Meaning
JMO	Judicial Medical Officer
FMD	Forensic Medical Department
MLR	Medico-Legal Report
PMR	Postmortem Report
NIC	National Identity Card
RBAC	Role-Based Access Control
CRUD	Create, Read, Update and Delete
UI	User Interface
RDBMS	Relational Database Management System
ACID	Atomicity, Consistency, Isolation and Durability
DDL	Data Definition Language
DML	Data Manipulation Language
DCL	Data Control Language
TCL	Transaction Control Language

Chapter 1 – Introduction

1.1 Background

Forensic medicine is a specialized branch of medical science that applies medical knowledge to legal investigations and judicial proceedings. Forensic Medical Departments are responsible for conducting medico-legal examinations, postmortem investigations, injury assessments, evidence collection, and the preparation of medico-legal reports requested by law enforcement agencies and courts. These departments play a significant role in ensuring justice by providing accurate medical evidence for criminal and civil cases.

Despite the importance of these services, many forensic medical departments continue to rely on manual record-keeping systems or isolated digital files. Patient information, police requests, examination records, postmortem findings, evidence details, and medico-legal reports are often maintained using paper documents, making record management time-consuming and error-prone. As the number of medico-legal cases increases, maintaining accurate, secure, and easily accessible records becomes increasingly difficult.

To overcome these challenges, this project proposes the design and development of a **Forensic Medical Database Management System**. The system provides a centralized database for managing patients, forensic cases, examinations, evidence, postmortem reports, medico-legal reports, police requests, doctors, and departmental information while ensuring data integrity, security, and efficient information retrieval.

1.2 Problem Statement

Most forensic medical departments still depend on manual or partially computerized record management systems. These traditional approaches introduce several operational challenges that affect efficiency, data accuracy, and security.

The major problems identified are:

- Dependence on paper-based records that are vulnerable to loss and physical damage.
- Difficulty in searching and retrieving historical patient and case records.
- Data redundancy and inconsistency due to duplicate record maintenance.
- Poor confidentiality and limited access control for sensitive medico-legal information.
- Delays in generating medico-legal and postmortem reports.
- Inefficient evidence management and chain-of-custody tracking.
- Difficulty in monitoring case progress and examination history.
- Lack of integration among departments such as police, doctors, and forensic administration.

These limitations reduce the overall efficiency of forensic medical services and increase the possibility of human error during legal investigations.

1.3 Objectives

General Objective

To design and develop a secure, reliable, and efficient **Forensic Medical Database Management System** that digitizes and manages medico-legal records, forensic cases, evidence, examinations, and report generation within a centralized database.

Specific Objectives

- Design a normalized relational database for forensic medical information.
- Maintain patient and forensic case records efficiently.
- Manage police requests and case registration.
- Store examination and postmortem findings securely.
- Track forensic evidence and maintain chain-of-custody information.
- Generate medico-legal and postmortem reports accurately.
- Improve data integrity through primary and foreign key constraints.
- Reduce data redundancy using database normalization techniques.
- Provide secure access to authorized users.
- Enable efficient searching and retrieval of forensic records.

1.4 Scope of the Project

The proposed system focuses on designing and implementing a database for the management of forensic medical information. The database includes modules for patient registration, doctor management, forensic case management, police requests, medical examinations, postmortem reports, evidence management, medico-legal reports, and departmental administration. It supports secure storage, retrieval, updating, and reporting of medico-legal data while maintaining relationships among all entities.

The project is limited to the design and implementation of the database and its supporting management functions. It does not include advanced technologies such as artificial intelligence, biometric authentication, mobile applications, cloud deployment, or integration with national healthcare and police information systems.

1.5 Significance of the Study

The implementation of a computerized forensic medical database significantly improves the management of medico-legal information. The proposed system enhances data accuracy, consistency, and security while reducing the dependency on manual documentation. It enables

faster retrieval of patient and case information, improves evidence tracking, simplifies report generation, and minimizes the risk of data loss. Furthermore, the database supports judicial and medical professionals by providing reliable and well-organized information for legal investigations. The system also demonstrates the practical application of database concepts such as Entity-Relationship modeling, normalization, relational schema design, integrity constraints, and SQL implementation. Therefore, this project serves as both a practical forensic information management solution and an educational database design project.

1.6 Technologies Used

The proposed **Forensic Medical Management System** was developed as a modern web-based application using frontend, backend, database, and authentication technologies. These technologies work together to provide a secure, responsive, and efficient platform for managing forensic medical information.

Technology	Purpose
React.js	Develops the interactive and responsive user interface using reusable components.
HTML5	Provides the structure and layout of web pages.
CSS3	Styles the application and enhances the user interface.
Bootstrap / Tailwind CSS	Creates responsive and visually appealing web pages across different devices.
JavaScript (ES6+)	Implements dynamic client-side functionality and application logic.
Firebase Authentication	Provides secure user authentication, login, logout, and user identity management.
Firebase Hosting (Optional)	Hosts the web application securely with fast global content delivery.
Node.js	Executes server-side JavaScript and handles backend operations.
Express.js	Develops RESTful APIs and manages communication between the frontend and database.
MySQL	Stores and manages forensic medical records using a relational database.
SQL	Defines, manipulates, and queries the database efficiently.
Git & GitHub	Supports version control and collaborative software development.
Draw.io (diagrams.net)	Designs Entity Relationship Diagrams (ERDs) and system diagrams.
Visual Studio Code	Integrated Development Environment (IDE) used for developing the application.

Technologies Overview

- **Frontend:** React.js, HTML5, CSS3, Bootstrap/Tailwind CSS, JavaScript
- **Backend:** Node.js, Express.js
- **Database:** MySQL
- **Authentication:** Firebase Authentication
- **Development Tools:** Visual Studio Code, Git, GitHub, Draw.io

This technology stack enables the development of a secure, scalable, and user-friendly forensic medical management system capable of efficiently handling patient records, forensic cases, evidence tracking, medical examinations, and report generation.

Chapter 2 – Requirement Analysis

2.1 Existing System

Describe the manual process.

Many forensic medical departments still rely on manual or semi-computerized methods to manage medico-legal information. Patient details, police requests, examination records, postmortem reports, evidence records, and medico-legal reports are commonly maintained using paper files, spreadsheets, or independent software systems that are not interconnected.

When a patient is referred by the police, clerical staff manually register patient details and create a case file. Doctors record examination findings and postmortem observations on paper documents, while evidence information is maintained separately. Preparing medico-legal reports requires searching through physical files, which consumes considerable time. Information sharing between doctors, police officers, and administrative staff is also inefficient because records are stored in different locations.

As the number of medico-legal cases increases, maintaining accurate records becomes increasingly difficult. Manual record keeping also increases the likelihood of misplaced documents, duplicate records, and delays in legal investigations.

2.2 Problems in Existing System

The current manual system suffers from several limitations:

- Paper-based records are vulnerable to damage, loss, and unauthorized access.

- Searching previous patient or case records is slow and time-consuming.
- Duplicate data entry causes redundancy and inconsistency.
- Evidence tracking is difficult due to manual documentation.
- Medico-legal report generation requires significant manual effort.
- Limited access control reduces information security.
- Communication between departments is inefficient.
- Difficult to monitor the progress of forensic cases.
- High probability of human errors during record management.
- Lack of centralized storage for forensic medical information.

2.3 Proposed System

Explain how the computerized system solves the problems.

The proposed **Forensic Medical Management System** is a secure web-based application that digitizes the entire forensic medical workflow. The system centralizes all medico-legal information into a single relational database, allowing authorized users to manage forensic cases efficiently.

The application enables patient registration, case creation, police request management, medical examinations, postmortem reporting, evidence tracking, and medico-legal report generation through an intuitive web interface. Firebase Authentication ensures that only authorized users can access the system based on their roles.

The proposed system minimizes paperwork, improves data accuracy, enhances security, simplifies report generation, and enables faster searching and retrieval of records. It also provides better evidence management and supports efficient collaboration among doctors, police officers, and administrators.

2.4 Stakeholders

The following stakeholders interact with the system:

Stakeholder	Responsibility
Patients	Provide personal information and undergo medico-legal examinations.
Judicial Medical Officers (JMOs)	Conduct examinations, prepare medico-legal reports, and manage forensic cases.
Doctors	Record examination findings and postmortem reports.
Police Officers	Submit police requests and monitor investigation progress.
Laboratory Staff	Manage evidence samples and laboratory test results.
Clerical Officers	Register patients and maintain administrative records.
Court Officials	Access finalized medico-legal reports for judicial proceedings.
System Administrator	Manage users, roles, permissions, and system maintenance.

2.5 Functional Requirements

The system shall provide the following functionalities:

User Management

- User Registration
- User Login
- Firebase Authentication
- Password Reset
- Role-Based Access Control
- Logout

Patient Management

- Register Patient
- Update Patient Information
- Search Patient Records
- View Patient History

Forensic Case Management

- Create New Case
- Assign Doctor
- Update Case Status
- View Case Details

Police Request Management

- Register Police Requests
- Track Request Status
- Assign Requests to Doctors

Examination Management

- Record Medical Examination Details
- Update Examination Results
- Store Injury Information

Postmortem Management

- Record Postmortem Findings
- Store Cause of Death
- Generate Postmortem Reports

Evidence Management

- Register Evidence
- Track Chain of Custody
- Update Evidence Status
- Search Evidence Records

Report Management

- Generate Medico-Legal Reports
- Generate Postmortem Reports
- Print Reports
- Download Reports

Search Module

- Search Patients
- Search Cases
- Search Reports
- Search Evidence

Administration

- Manage Users

- Manage Departments
- Manage Doctors
- Monitor System Activities

2.6 Non-functional Requirements

Security

- Firebase Authentication shall securely authenticate users.
- Passwords and sensitive information shall be securely managed.
- Only authorized users shall access restricted information.

Performance

- Search operations should complete within a few seconds.
- The system should efficiently manage a large number of records.

Availability

- The application should be available whenever authorized users require access.

Confidentiality

- Patient and medico-legal records shall only be accessible by authorized personnel.

Reliability

- The system shall maintain data integrity during all database transactions.
- Backup mechanisms should prevent accidental data loss.

Usability

- The user interface should be simple, responsive, and easy to learn.
- Users should complete common tasks with minimal effort.

Scalability

- The system should support future expansion, including additional modules and users.

Maintainability

- The application should be modular to simplify updates and maintenance.

2.7 Use Cases

Use Case Diagram

Actors

- System Administrator
- Doctor / Judicial Medical Officer
- Police Officer
- Laboratory Staff
- Clerical Officer
- Court Official

Main Use Cases

- Login
- Register Patient
- Create Case
- Manage Police Requests
- Record Examination
- Record Postmortem
- Manage Evidence
- Generate Reports
- Search Records
- Manage Users
- Logout

Use Case Descriptions

Use Case UC-01 – User Login

Item	Description
Actor	All Users
Description	Users log into the system using Firebase Authentication.
Preconditions	User account exists.
Main Flow	Enter email → Enter password → Authentication → Dashboard
Postconditions	User gains access according to assigned role.

Use Case UC-02 – Register Patient

Item	Description
Actor	Clerical Officer
Description	Registers a new patient into the system.
Preconditions	User is authenticated.
Main Flow	Enter patient details → Validate → Save to database
Postconditions	Patient record is created.

Use Case UC-03 – Create Forensic Case

Item	Description
Actor	Doctor / JMO
Description	Creates a new forensic case for a patient.
Preconditions	Patient already registered.
Main Flow	Select patient → Enter case details → Save
Postconditions	New case is created.

Use Case UC-04 – Manage Evidence

Item	Description
Actor	Laboratory Staff

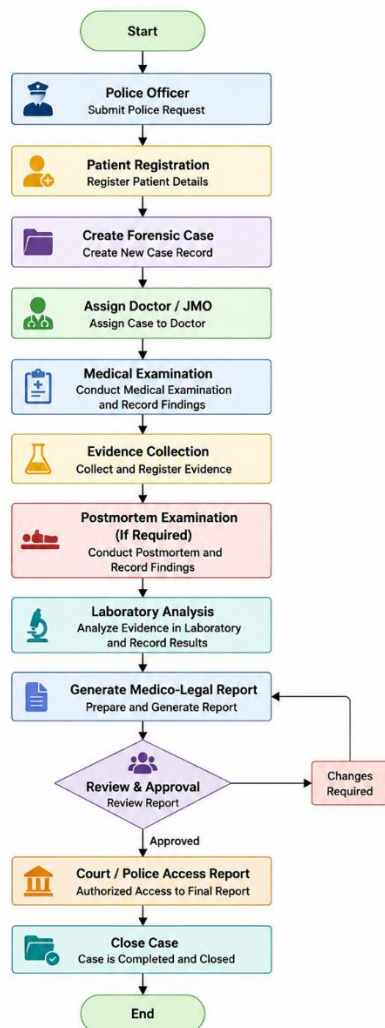
Item	Description
Description	Registers and tracks forensic evidence.
Preconditions	Case exists.
Main Flow	Add evidence → Store location → Update status
Postconditions	Evidence information is securely stored.

Use Case UC-05 – Generate Reports

Item	Description
Actor	Doctor
Description	Generates medico-legal and postmortem reports.
Preconditions	Examination completed.
Main Flow	Select case → Generate report → Save/Print
Postconditions	Report becomes available for authorized users.

2.8 Workflow / Business Process

Flowchart



Chapter 3 – Database Design

3.1 Entity Identification

The following entities were identified during the analysis and design of the **Forensic Medical Management System**. These entities represent the major objects involved in managing forensic medical records and their relationships within the system.

- Department
- Doctor
- Patient
- Forensic Case
- Police Officer
- Police Request
- Examination
- Evidence
- Postmortem Report
- Medico-Legal Report

These entities form the core components of the database and are interconnected through various relationships to support patient registration, case management, medical examinations, evidence tracking, report generation, and departmental administration.

If your lecturer expects only the entity names (like the example), simply write:

3.1 Entity Identification

- Department
- Doctor
- Patient
- Forensic Case
- Police Officer
- Police Request
- Examination
- Evidence
- Postmortem Report
- Medico-Legal Report

3.2 ER Diagram

Include complete ERD.



3.3 Enhanced ER Diagram (Optional)

CASE

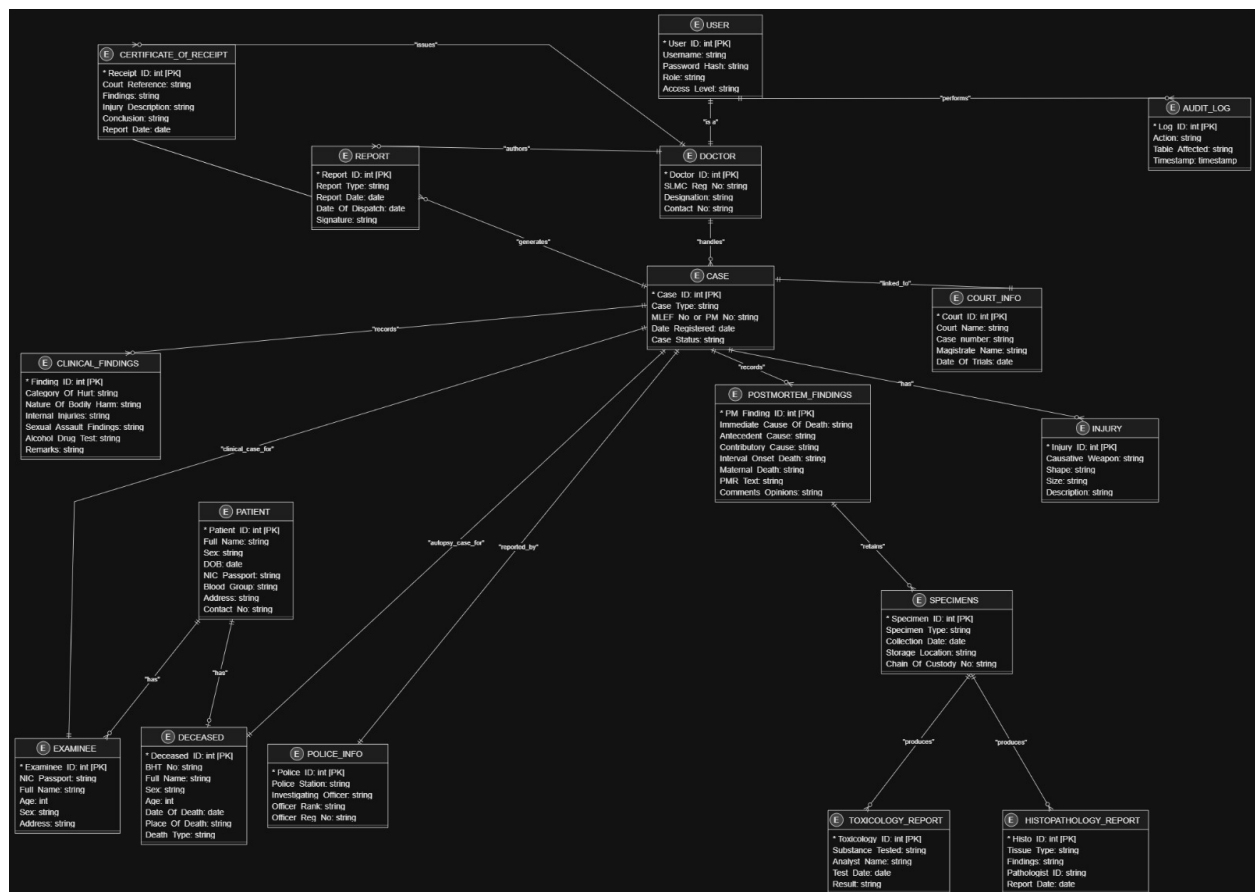
PK Case_ID

Case_Type

MLEF_No_or_PM_No

Date_Registered

Case_Status



3.4 Relational Schema

Include all tables.

POLICE_INFO

PK Police_Info_ID

FK Case_ID

Police_Station

Officer_Name

Badge_No

COURT_INFO

PK Court_Info_ID

FK Case_ID

Court_Name

Magistrate

Sex

Age

INVESTIGATION

PK Investigation_ID

FK Case_ID

Investigation_Date

Remarks

FK Hospital_ID

FK Ward_ID

Full_Name

Sex

Age

DOCUMENT

CLINICAL_FINDINGS

PK Clinical_Finding_ID

FK Case_ID

Findings

INJURY

PK Injury_ID

FK Case_ID

Injury_Type

Location

Description

REPORT

PK Report_ID

FK Case_ID

FK Doctor_ID

Report_Type

Report_Date

EXHIBIT

PK Exhibit_ID

FK Case_ID

FK Police_Info_ID

Exhibit_Type

Description

USER

PK User_ID

Username

Password_Hash

Role

Access_Level

DOCTOR

PK Doctor_ID

FK User_ID

Name

SLMC_Reg_No

Designation

POSTMORTEM_FINDINGS

PK PM_Finding_ID

FK Case_ID

Findings

Cause_of_Death

CONSENT

PK Consent_ID

FK Examinee_ID

Consent_Date

NEXT_OF_KIN

PK Next_of_Kin_ID

FK Deceased_ID

Name

Relationship

Contact_No

BODY_IDENTIFICATION

PK Identification_ID

FK Deceased_ID

FK Next_of_Kin_ID

Identification_Date

SPECIMENS

PK Specimen_ID

FK PM_Finding_ID

Specimen_Type

HOSPITAL

PK Hospital_ID

Hospital_Name

Address

WARD

PK Ward_ID

FK Hospital_ID

Ward_Name

CERTIFICATE_OF_RECEIPT

PK Receipt_ID

FK Doctor_ID

FK Report_ID

Issue_Date

3.5 Data Dictionary

The data dictionary provides a detailed description of each table in the database, including its attributes, data types, sizes, descriptions, and constraints. It serves as a reference for

HISTOPATHOLOGY_REPORT

PK Histopathology_Report_ID

FK Doctor_ID

FK Specimen_ID

Report_Date

TOXICOLOGY_REPORT

PK Toxicology_Report_ID

FK Specimen_ID

Report_Date

understanding the database structure and ensures consistency during implementation and maintenance.

Table 3.12 – Department

Attribute	Data Type	Size	Description	Constraints
Department_ID	INT	11	Unique department identifier	Primary Key, Auto Increment
Department_Name	VARCHAR	100	Name of the department	NOT NULL
HeadDoctor_ID	INT	11	Head doctor of the department	Foreign Key
ContactNo	VARCHAR	15	Department contact number	NOT NULL
Location	VARCHAR	100	Department location	NOT NULL

Table 3.13 – Doctor

Attribute	Data Type	Size	Description	Constraints
Doctor_ID	INT	11	Unique doctor identifier	Primary Key, Auto Increment
FirstName	VARCHAR	50	Doctor's first name	NOT NULL
LastName	VARCHAR	50	Doctor's last name	NOT NULL
NIC	VARCHAR	12	National Identity Card number	UNIQUE, NOT NULL
Email	VARCHAR	100	Doctor email address	UNIQUE
Contact_No	VARCHAR	15	Contact number	NOT NULL
Registration_No	VARCHAR	30	SLMC Registration Number	UNIQUE
Specialization	VARCHAR	100	Medical specialization	NOT NULL

Department_ID	INT	11	Department identifier	Foreign Key
---------------	-----	----	-----------------------	-------------

Table 3.14 – Patient

Attribute	Data Type	Size	Description	Constraints
Patient_ID	INT	11	Unique patient identifier	Primary Key, Auto Increment
First_Name	VARCHAR	50	First name	NOT NULL
Last_Name	VARCHAR	50	Last name	NOT NULL
NIC	VARCHAR	12	National Identity Card	UNIQUE
Gender	ENUM	-	Gender	NOT NULL
DOB	DATE	-	Date of birth	NOT NULL
Blood_Group	VARCHAR	5	Blood group	NULL
Address	VARCHAR	255	Residential address	NOT NULL
ContactNo	VARCHAR	15	Contact number	NOT NULL

Table 3.15 – Forensic_Case

Attribute	Data Type	Size	Description	Constraints
Case_ID	INT	11	Unique case identifier	Primary Key, Auto Increment
Patient_ID	INT	11	Related patient	Foreign Key
Doctor_ID	INT	11	Assigned doctor	Foreign Key
Case_Type	VARCHAR	50	Type of forensic case	NOT NULL
Case_Status	VARCHAR	30	Current status	DEFAULT 'Open'
Date_Registered	DATE	-	Registration date	NOT NULL

Table 3.16 – Police_Officer

Attribute	Data Type	Size	Description	Constraints
Officer_ID	INT	11	Unique officer identifier	Primary Key
First_Name	VARCHAR	50	First name	NOT NULL
Last_Name	VARCHAR	50	Last name	NOT NULL
Police_Station	VARCHAR	100	Police station	NOT NULL
Rank	VARCHAR	50	Officer rank	NOT NULL
Email	VARCHAR	100	Email address	UNIQUE

Table 3.17 – Police_Request

Attribute	Data Type	Size	Description	Constraints
Request_ID	INT	11	Unique request identifier	Primary Key
Officer_ID	INT	11	Police officer	Foreign Key
Case_ID	INT	11	Related forensic case	Foreign Key
RequestDate	DATE	-	Date of request	NOT NULL
Purpose	TEXT	-	Purpose of request	NOT NULL
Status	VARCHAR	30	Request status	DEFAULT 'Pending'

Table 3.18 – Examination

Attribute	Data Type	Size	Description	Constraints
Examination_ID	INT	11	Examination identifier	Primary Key
Case_ID	INT	11	Related case	Foreign Key
Doctor_ID	INT	11	Conducting doctor	Foreign Key
Examination_Type	VARCHAR	100	Type of examination	NOT NULL
Examination_Date	DATE	-	Examination date	NOT NULL
Result	TEXT	-	Examination result	NULL
Observations	TEXT	-	Doctor observations	NULL

Table 3.19 – Evidence

Attribute	Data Type	Size	Description	Constraints
Evidence_ID	INT	11	Unique evidence identifier	Primary Key

Case_ID	INT	11	Related forensic case	Foreign Key
Evidence_Type	VARCHAR	100	Type of evidence	NOT NULL
Description	TEXT	-	Evidence description	NULL
ChainOfCustody	TEXT	-	Chain of custody information	NOT NULL
Storage_Location	VARCHAR	100	Evidence storage location	NOT NULL
Collected_By	VARCHAR	100	Officer who collected evidence	NOT NULL
Collected_Date	DATE	-	Date collected	NOT NULL

Table 3.20 – Medico_Legal_Report

Attribute	Data Type	Size	Description	Constraints
Report_ID	INT	11	Unique report identifier	Primary Key
Case_ID	INT	11	Related forensic case	Foreign Key
Doctor_ID	INT	11	Report author	Foreign Key
Report_Date	DATE	-	Date generated	NOT NULL
Findings	TEXT	-	Medical findings	NOT NULL
Injury_Description	TEXT	-	Injury details	NULL
Conclusion	TEXT	-	Final conclusion	NOT NULL
Court_Reference	VARCHAR	100	Court reference number	NULL

Table 3.21 – Postmortem_Report

Attribute	Data Type	Size	Description	Constraints
Report_ID	INT	11	Unique report identifier	Primary Key
Case_ID	INT	11	Related forensic case	Foreign Key
Doctor_ID	INT	11	Doctor conducting postmortem	Foreign Key
Examination_Date	DATE	-	Examination date	NOT NULL
CauseOfDeath	VARCHAR	255	Cause of death	NOT NULL
Findings	TEXT	-	Postmortem findings	NOT NULL
MannerOfDeath	VARCHAR	100	Manner of death	NOT NULL
BodyCondition	TEXT	-	Physical condition of body	NULL

3.6 Primary Keys

Table	Primary Key
Department	Department_ID
Doctor	Doctor_ID
Patient	Patient_ID
Forensic_Case	Case_ID
Police_Officer	Officer_ID
Police_Request	Request_ID
Examination	Examination_ID
Evidence	Evidence_ID
Medico_Legal_Report	Report_ID
Postmortem_Report	Report_ID

3.7 Foreign Keys

Table	Foreign Key	References
Doctor	Department_ID	Department(Department_ID)
Forensic_Case	Patient_ID	Patient(Patient_ID)
Forensic_Case	Doctor_ID	Doctor(Doctor_ID)
Police_Request	Officer_ID	Police_Officer(Officer_ID)
Police_Request	Case_ID	Forensic_Case(Case_ID)
Examination	Case_ID	Forensic_Case(Case_ID)
Examination	Doctor_ID	Doctor(Doctor_ID)
Evidence	Case_ID	Forensic_Case(Case_ID)
Medico_Legal_Report	Case_ID	Forensic_Case(Case_ID)
Medico_Legal_Report	Doctor_ID	Doctor(Doctor_ID)
Postmortem_Report	Case_ID	Forensic_Case(Case_ID)
Postmortem_Report	Doctor_ID	Doctor(Doctor_ID)

3.8 Integrity Constraints

Entity Integrity

Every table contains a primary key that uniquely identifies each record.

Examples:

- Department_ID
- Doctor_ID
- Patient_ID
- Case_ID
- Evidence_ID

These attributes cannot be NULL or duplicated.

Referential Integrity

Foreign keys ensure that relationships between tables remain valid.

Examples:

- A Doctor must belong to an existing Department.
- A Forensic Case must belong to an existing Patient.
- An Examination cannot exist unless the corresponding Case exists.
- An Evidence record must be associated with an existing Case.
- Reports can only be generated for valid forensic cases.

If a referenced record does not exist, the database rejects the operation.

Domain Integrity

Attributes are restricted to valid data values.

Examples include:

- Gender = ('Male','Female','Other')
- Case_Status = ('Open','Pending','Closed')
- Blood_Group = ('A+','A-','B+','B-','AB+','AB-','O+','O-')
- Examination_Date cannot be NULL.
- Request_Date must be a valid DATE.
- Email must follow a valid email format.

NOT NULL Constraints

Important attributes must always contain values.

Examples:

- Patient Name
- NIC
- Doctor Name
- Registration Number
- Report Date
- Examination Date

UNIQUE Constraints

The following values should be unique:

- Patient NIC
- Doctor Registration Number
- Police Officer Email
- Department Name

CHECK Constraints

Examples include:

- Age > 0
- Contact Number contains only digits
- Report Date ≥ Examination Date
- Collection Date cannot be in the future

These integrity constraints help maintain data quality while reducing data inconsistency and invalid records.

3.9 Normalization

The database was normalized to **Third Normal Form (3NF)** to eliminate redundancy and improve consistency.

Unnormalized Form (UNF)

Initially, all patient, doctor, case, examination, evidence, and report information could be stored in a single large table.

Example:

Patient	Doctor	Case	Evidence	Report
John	Dr. Silva	C001	Blood Sample	MLR001

Problems:

- Data duplication
- Multiple repeating groups
- Difficult updates
- Data inconsistency

First Normal Form (1NF)

The repeating groups were removed, and each attribute contains only a single (atomic) value.

Separate tables were created for:

- Patient
- Doctor
- Department
- Forensic_Case
- Police_Officer
- Police_Request
- Examination
- Evidence
- Medico_Legal_Report
- Postmortem_Report

Each table has a unique primary key.

Second Normal Form (2NF)

Partial dependencies were removed by ensuring that all non-key attributes depend entirely on the primary key.

Examples:

- Doctor information depends only on Doctor_ID.
- Patient information depends only on Patient_ID.
- Evidence details depend only on Evidence_ID.
- Examination details depend only on Examination_ID.

This eliminates redundant storage of doctor and patient information.

Third Normal Form (3NF)

Transitive dependencies were removed.

Examples:

- Department details are stored only in the Department table.
- Doctor records reference Department_ID instead of repeating department information.
- Police Officer information is stored separately and referenced by Police_Request.
- Reports reference Case_ID and Doctor_ID instead of storing duplicate patient and doctor details.

After normalization:

- Data redundancy is minimized.
- Data consistency is improved.
- Insert, update, and delete anomalies are eliminated.
- Database maintenance becomes easier.
- Storage efficiency is improved.

Chapter 4 – Database Implementation

4.1 DBMS Used

The Forensic Medical Management System was implemented using **MySQL**, a widely used open-source Relational Database Management System (RDBMS). MySQL was selected because of its reliability, scalability, security, and support for Structured Query Language (SQL). It provides efficient mechanisms for creating, storing, retrieving, and managing relational data while enforcing integrity constraints.

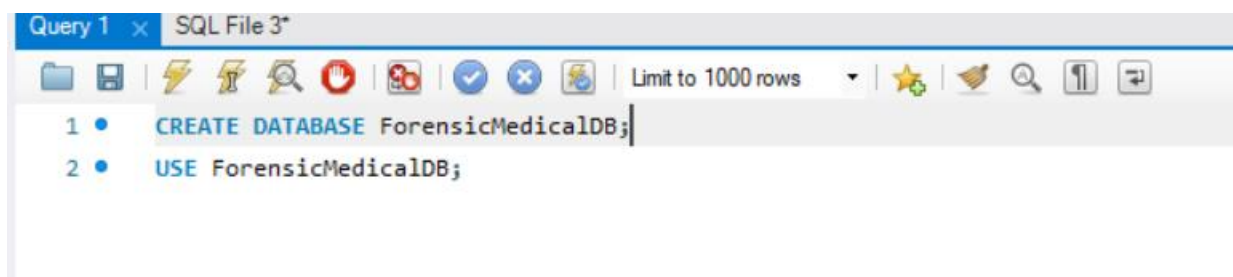
The database was developed using **MySQL Workbench**, which provides graphical tools for database modeling, SQL script execution, and database administration. SQL was used to define database structures, create relationships, manipulate data, and retrieve information.

Features of MySQL used in this project

- Support for relational database design
- Primary key and foreign key constraints
- Data integrity enforcement
- Indexing for faster searching
- Views for simplified data retrieval
- Stored procedures for reusable database operations
- Triggers for automatic database actions
- ACID-compliant transactions
- Backup and recovery support

4.2 Database Creation

The database was created using SQL Data Definition Language (DDL) commands. First, a dedicated database named **ForensicMedicalDB** was created and selected for use.



After creating the database, individual tables were created according to the relational schema developed in Chapter 3. Relationships between entities were established using foreign key constraints to ensure referential integrity.

The database implementation follows the normalized design up to Third Normal Form (3NF), reducing redundancy while maintaining consistency among related records.

4.3 Table Creation

The tables were created using SQL **CREATE TABLE** statements. Each table contains appropriate data types, primary keys, foreign keys, and additional constraints.

```
1 • CREATE TABLE Department (  
2     Department_ID INT AUTO_INCREMENT PRIMARY KEY,  
3     Department_Name VARCHAR(100) NOT NULL,  
4     ContactNo VARCHAR(15),  
5     Location VARCHAR(100)  
6 );  
7 • CREATE TABLE Doctor (  
8     Doctor_ID INT AUTO_INCREMENT PRIMARY KEY,  
9     FirstName VARCHAR(50),  
10    LastName VARCHAR(50),  
11    Registration_No VARCHAR(30) UNIQUE,  
12    Email VARCHAR(100),  
13    Department_ID INT,  
14    FOREIGN KEY (Department_ID)  
15        REFERENCES Department(Department_ID)  
16 );  
17  
18 • CREATE TABLE Patient (  
19     Patient_ID INT AUTO_INCREMENT PRIMARY KEY,  
20     First_Name VARCHAR(50) NOT NULL,  
21     Last_Name VARCHAR(50) NOT NULL,  
22     NIC VARCHAR(12) UNIQUE NOT NULL,  
23     Gender ENUM('Male','Female','Other') NOT NULL,  
24     DOB DATE NOT NULL,  
25     Blood_Group VARCHAR(5),  
26     Address VARCHAR(255),  
27     ContactNo VARCHAR(15)  
28 );
```

```

29 • ○ CREATE TABLE Forensic_Case (
30     Case_ID INT AUTO_INCREMENT PRIMARY KEY,
31     Patient_ID INT NOT NULL,
32     Doctor_ID INT NOT NULL,
33     Case_Type VARCHAR(100),
34     Date_Registered DATE NOT NULL,
35     Case_Status VARCHAR(30) DEFAULT 'Open',
36
37     FOREIGN KEY (Patient_ID)
38     REFERENCES Patient(Patient_ID),
39
40     FOREIGN KEY (Doctor_ID)
41     REFERENCES Doctor(Doctor_ID)
42 );

```

```

43 • ○ CREATE TABLE Police_Officer (
44     Officer_ID INT AUTO_INCREMENT PRIMARY KEY,
45     First_Name VARCHAR(50) NOT NULL,
46     Last_Name VARCHAR(50) NOT NULL,
47     Police_Station VARCHAR(100),
48 ✖ Rank VARCHAR(50),
49     Email VARCHAR(100) UNIQUE
50 );
51 • ○ CREATE TABLE Police_Request (
52     Request_ID INT AUTO_INCREMENT PRIMARY KEY,
53     Officer_ID INT,
54     Case_ID INT,
55     RequestDate DATE NOT NULL,
56     Purpose TEXT,
57     Status VARCHAR(30) DEFAULT 'Pending',
58
59     FOREIGN KEY (Officer_ID)
60     REFERENCES Police_Officer(Officer_ID),
61
62     FOREIGN KEY (Case_ID)
63     REFERENCES Forensic_Case(Case_ID)
64 );

```

```
65 • CREATE TABLE Police_Request (  
66     Request_ID INT AUTO_INCREMENT PRIMARY KEY,  
67     Officer_ID INT,  
68     Case_ID INT,  
69     RequestDate DATE NOT NULL,  
70     Purpose TEXT,  
71     Status VARCHAR(30) DEFAULT 'Pending',  
72     FOREIGN KEY (Officer_ID)  
73     REFERENCES Police_Officer(Officer_ID),  
74     FOREIGN KEY (Case_ID)  
75     REFERENCES Forensic_Case(Case_ID)  
76 );  
77 • CREATE TABLE Examination (  
78     Examination_ID INT AUTO_INCREMENT PRIMARY KEY,  
79     Case_ID INT,  
80     Doctor_ID INT,  
81     Examination_Type VARCHAR(100),  
82     Examination_Date DATE,  
83     Result TEXT,  
84     Observations TEXT,  
85     FOREIGN KEY (Case_ID)  
86     REFERENCES Forensic_Case(Case_ID),
```

```
CREATE TABLE Evidence (  
    Evidence_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Case_ID INT,  
    Evidence_Type VARCHAR(100),  
    Description TEXT,  
    ChainOfCustody TEXT,  
    Storage_Location VARCHAR(100),  
    Collected_By VARCHAR(100),  
    Collected_Date DATE,  
  
    FOREIGN KEY (Case_ID)  
    REFERENCES Forensic_Case(Case_ID)  
);
```

```
CREATE TABLE Medico_Legal_Report (  
    Report_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Case_ID INT,  
    Doctor_ID INT,  
    Report_Date DATE,  
    Findings TEXT,  
    Injury_Description TEXT,  
    Conclusion TEXT,
```

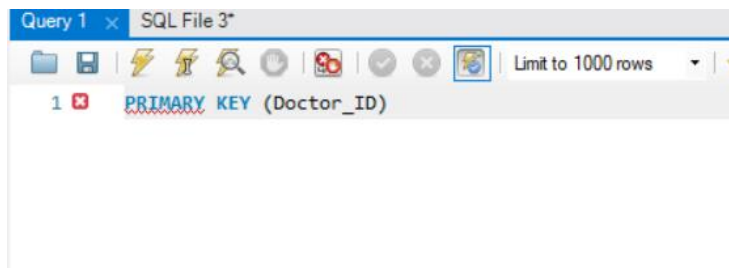
```
CREATE TABLE Postmortem_Report (  
    Report_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Case_ID INT,  
    Doctor_ID INT,  
    Examination_Date DATE,  
    CauseOfDeath VARCHAR(255),  
    Findings TEXT,  
    MannerOfDeath VARCHAR(100),  
    BodyCondition TEXT,  
  
    FOREIGN KEY (Case_ID)  
    REFERENCES Forensic_Case(Case_ID),  
  
    FOREIGN KEY (Doctor_ID)  
    REFERENCES Doctor(Doctor_ID)  
);
```

4.4 Constraints

To ensure database integrity and consistency, several SQL constraints were applied.

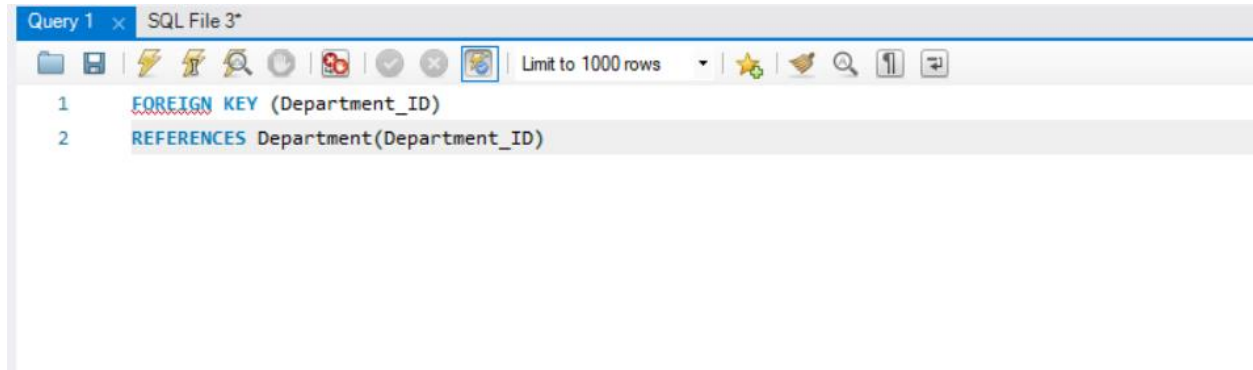
Primary Key

Every table contains a primary key.



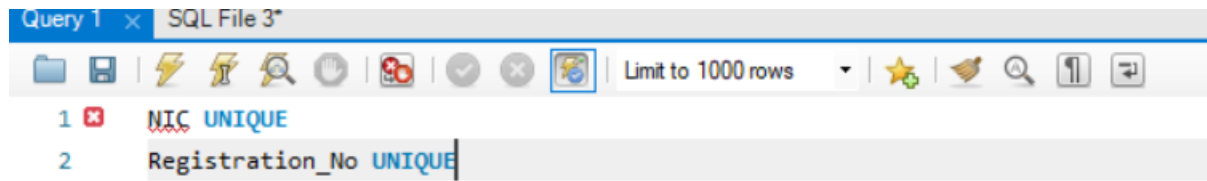
Foreign Key

Relationships between entities are maintained using foreign keys.



UNIQUE

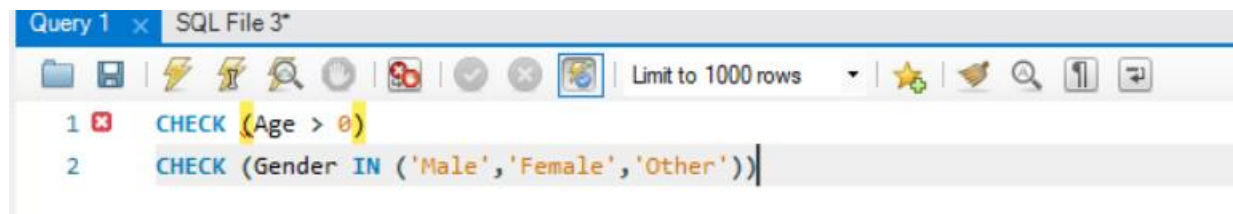
Used to prevent duplicate values.



CHECK

Used to validate attribute values.

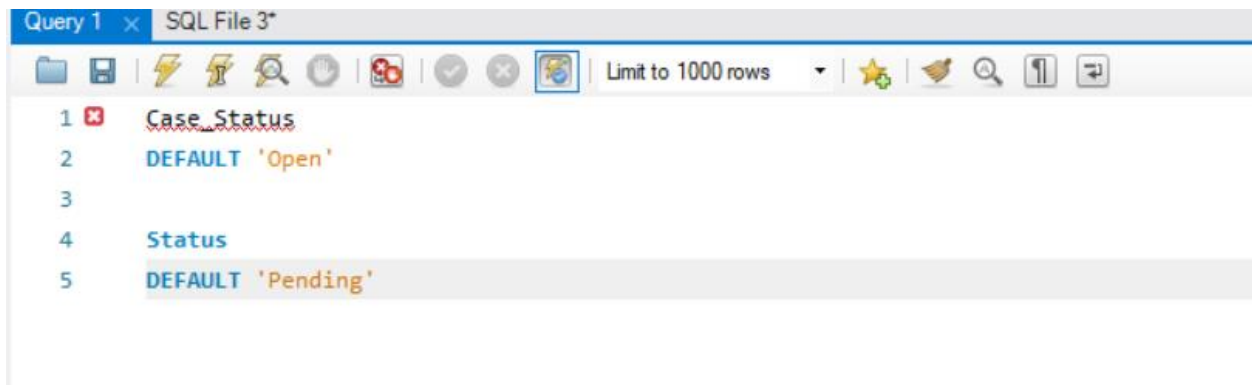
Example



DEFAULT

Provides default values.

Example



The screenshot shows a SQL IDE window titled 'Query 1' and 'SQL File 3*'. The toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The query text is as follows:

```
1 Case_Status
2 DEFAULT 'Open'
3
4 Status
5 DEFAULT 'Pending'
```

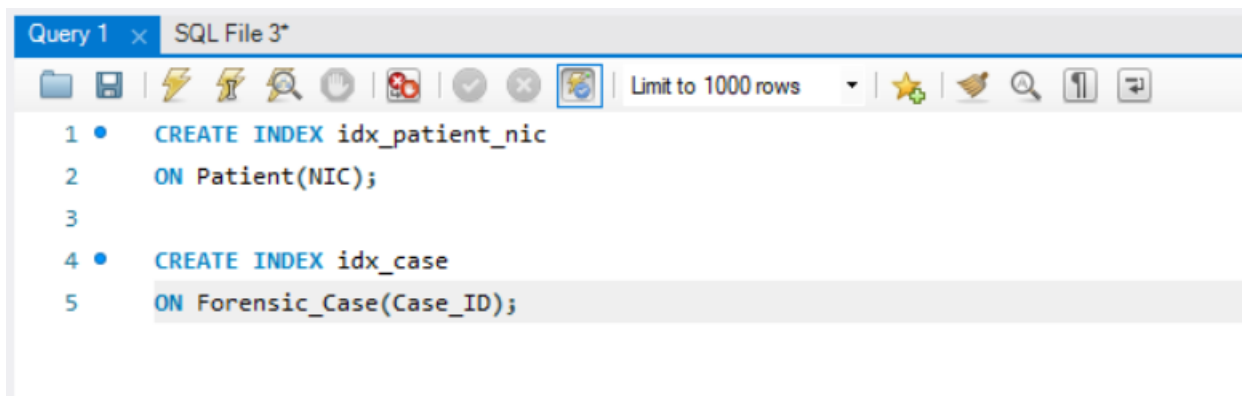
These constraints improve data quality and prevent invalid records from being inserted into the database.

4.5 Indexes

Indexes were created on frequently searched attributes to improve query performance.

Examples include:

Table	Indexed Attribute
Patient	NIC
Doctor	Registration_No
Forensic_Case	Case_ID
Evidence	Evidence_Type
Police_Request	RequestDate



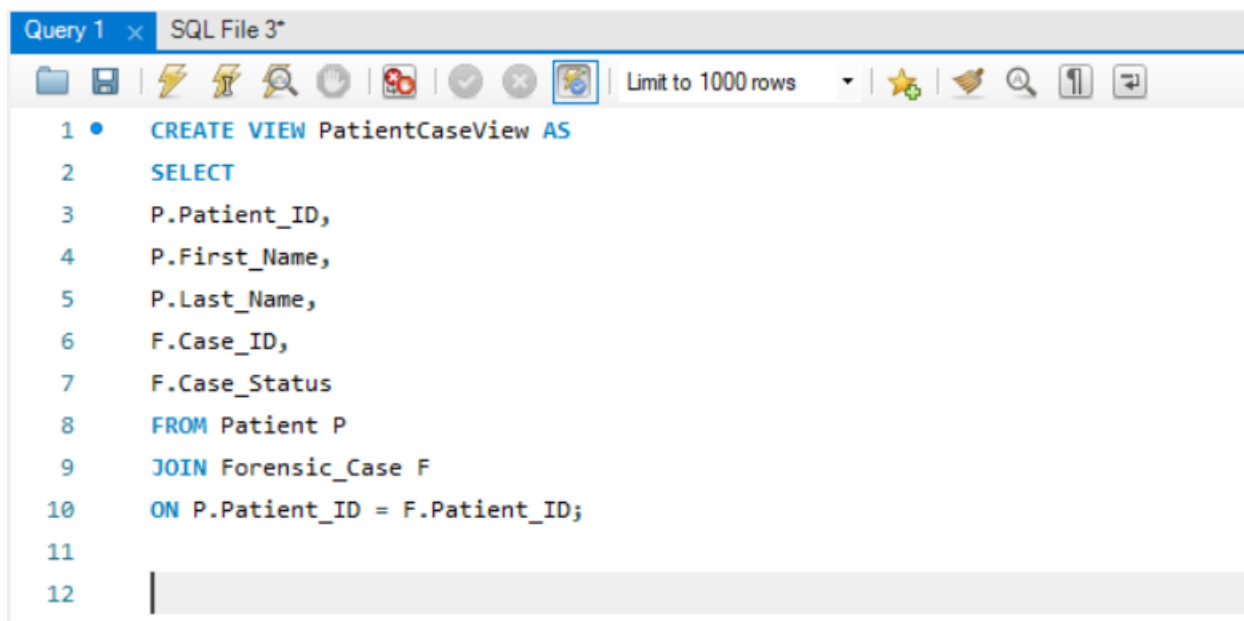
The screenshot shows a SQL IDE window titled 'Query 1' and 'SQL File 3*'. The toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The query text is as follows:

```
1 CREATE INDEX idx_patient_nic
2 ON Patient(NIC);
3
4 CREATE INDEX idx_case
5 ON Forensic_Case(Case_ID);
```

4.6 Views

Views simplify complex SQL queries by presenting frequently required information through virtual tables.

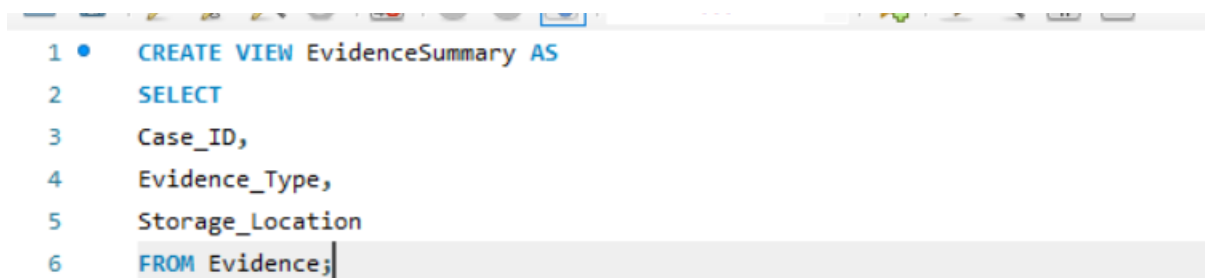
Patient Case View



The screenshot shows a SQL query editor window titled "Query 1" and "SQL File 3*". The toolbar includes icons for file operations, execution, and a "Limit to 1000 rows" dropdown. The query text is as follows:

```
1 • CREATE VIEW PatientCaseView AS
2 SELECT
3   P.Patient_ID,
4   P.First_Name,
5   P.Last_Name,
6   F.Case_ID,
7   F.Case_Status
8 FROM Patient P
9 JOIN Forensic_Case F
10 ON P.Patient_ID = F.Patient_ID;
11
12
```

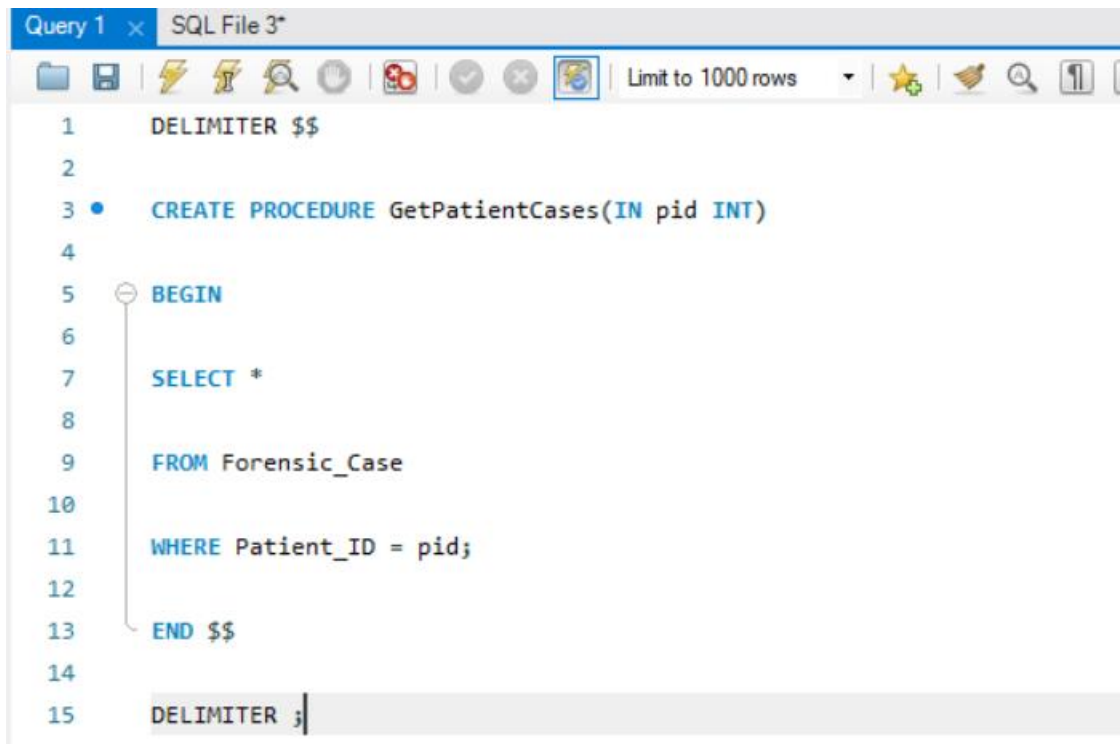
Evidence Summary View



The screenshot shows a SQL query editor window with a toolbar. The query text is as follows:

```
1 • CREATE VIEW EvidenceSummary AS
2 SELECT
3   Case_ID,
4   Evidence_Type,
5   Storage_Location
6 FROM Evidence;
```

4.7 Stored Procedures

A screenshot of a SQL IDE window titled "Query 1" and "SQL File 3*". The window contains a SQL script for creating a stored procedure. The script is as follows:

```
1 DELIMITER $$
2
3 CREATE PROCEDURE GetPatientCases(IN pid INT)
4
5 BEGIN
6
7 SELECT *
8
9 FROM Forensic_Case
10
11 WHERE Patient_ID = pid;
12
13 END $$
14
15 DELIMITER ;
```

The script is displayed with line numbers from 1 to 15 on the left. A toolbar at the top includes icons for file operations, execution, and a "Limit to 1000 rows" dropdown.

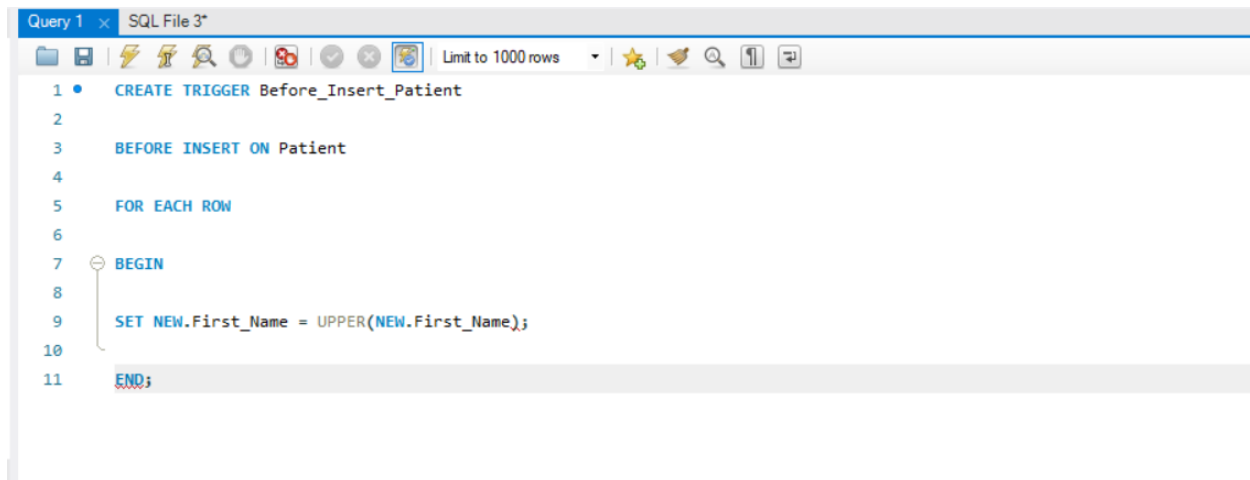
Another procedure may be used to retrieve all evidence associated with a particular forensic case.

Stored procedures improve maintainability while reducing application-side SQL complexity.

4.8 Triggers

Triggers automatically execute predefined SQL statements whenever specified database events occur.

Example

A screenshot of a SQL IDE window titled 'Query 1' and 'SQL File 3*'. The window contains a SQL script for creating a trigger. The script is as follows:

```
1 • CREATE TRIGGER Before_Insert_Patient
2
3 BEFORE INSERT ON Patient
4
5 FOR EACH ROW
6
7 BEGIN
8
9 SET NEW.First_Name = UPPER(NEW.First_Name);
10
11 END;
```

The script is numbered 1 through 11. The IDE interface includes a toolbar with various icons and a status bar at the bottom indicating 'Limit to 1000 rows'.

Another trigger can automatically update audit logs whenever medico-legal reports are generated.

Triggers help automate repetitive tasks and improve database consistency.

4.9 Sample Data

After creating the tables, sample records were inserted to verify database functionality.

Example Department record

Department_ID	Department_Name	Location
---------------	-----------------	----------

1	Forensic Medicine	Peradeniya
---	-------------------	------------

Example Doctor record

Doctor_ID	Name	Specialization
1	Dr. Nimal Perera	Forensic Medicine

Example Patient record

Patient_ID	Name	NIC
1	Kasun Silva	200012345678

Example Case record

Case_ID	Patient_ID	Status
1001	1	Open

Example Evidence record

Evidence_ID	Case_ID	Evidence_Type
1	1001	Blood Sample

Sample data were used to test:

- table relationships
 - SQL queries
 - report generation
 - joins
 - foreign key constraints
 - CRUD operations
- The Forensic Medical Management System was developed using a three-tier architecture consisting of the presentation layer, application layer, and database layer. The frontend was implemented using React.js, HTML5, CSS3, and Bootstrap to provide a responsive user interface. The backend was developed using Node.js and Express.js to handle business logic and API requests. MySQL serves as the relational database for storing all forensic medical records. Firebase Authentication is integrated to provide secure user authentication and role-based access control.
- The overall workflow is as follows:
 - Users access the web application through a browser.
 - Firebase Authentication verifies user credentials.
 - The frontend sends API requests to the Express.js backend.
 - The backend processes requests and communicates with the MySQL database.
 - Retrieved data are returned to the frontend and displayed to the user.
 -

Chapter 5 – System Development

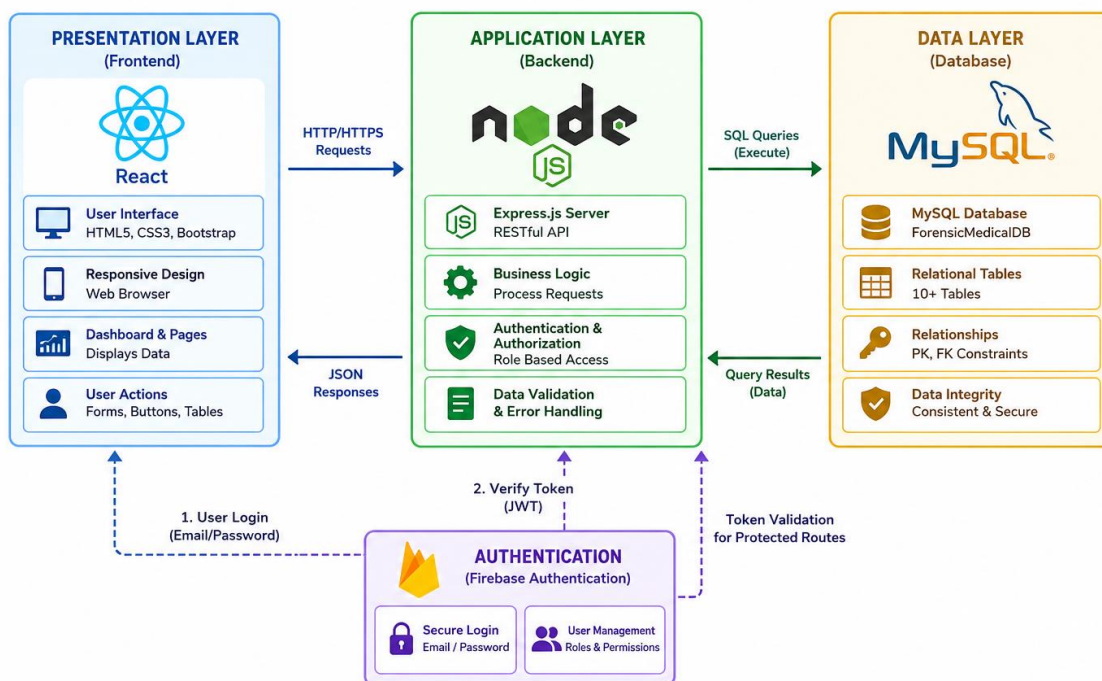
5.1 System Architecture

The Forensic Medical Management System was developed using a three-tier architecture consisting of the presentation layer, application layer, and database layer. The frontend was implemented using React.js, HTML5, CSS3, and Bootstrap to provide a responsive user interface. The backend was developed using Node.js and Express.js to handle business logic and API requests. MySQL serves as the relational database for storing all forensic medical records. Firebase Authentication is integrated to provide secure user authentication and role-based access control.

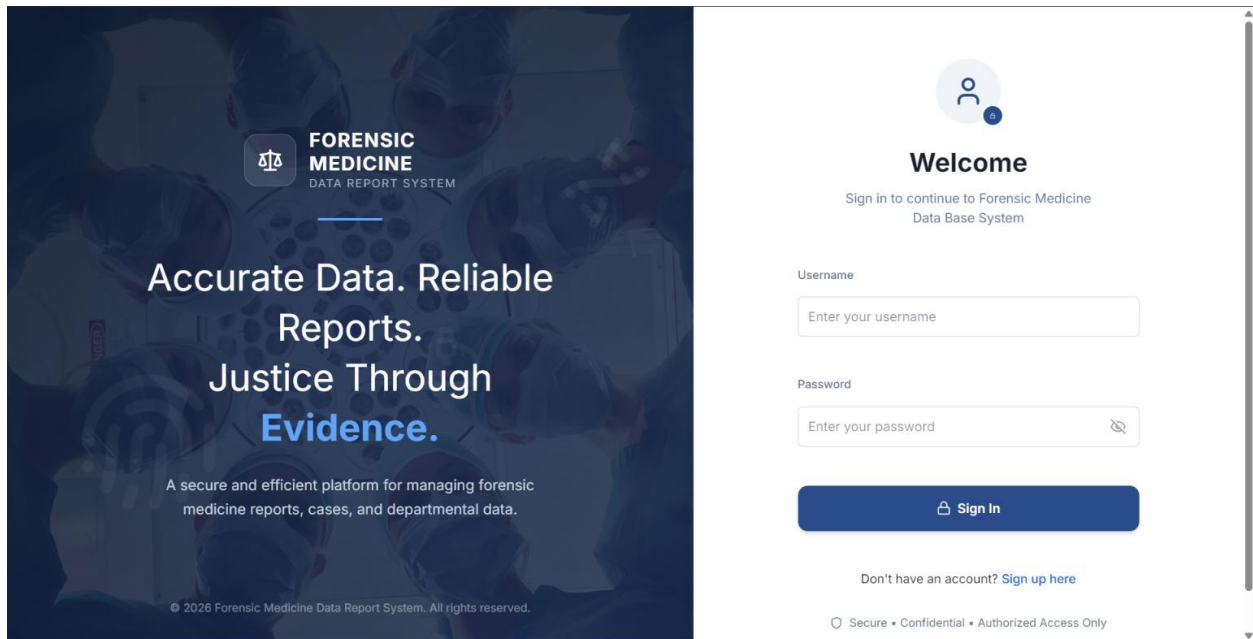
The overall workflow is as follows:

1. Users access the web application through a browser.
2. Firebase Authentication verifies user credentials.
3. The frontend sends API requests to the Express.js backend.
4. The backend processes requests and communicates with the MySQL database.
5. Retrieved data are returned to the frontend and displayed to the user.

Forensic Medical Management System System Architecture



5.2 Login Module



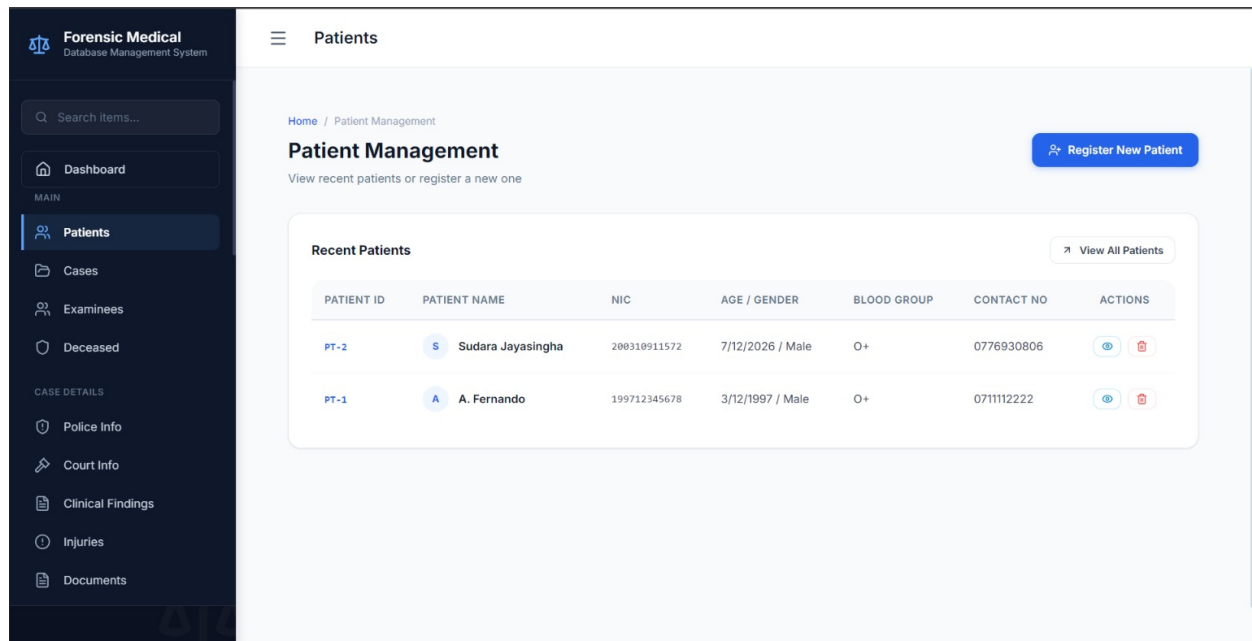
The Login Module is the entry point of the application. Users authenticate using Firebase Authentication with their registered email address and password. After successful authentication, users are redirected to their respective dashboards based on their assigned roles.

The module validates user credentials before granting access to the system. Unauthorized users are prevented from accessing protected resources.

Main Features

- User Login
- Email and Password Authentication
- Firebase Authentication
- Password Reset
- Logout
- Error Messages for Invalid Login

5.3 Patient Management



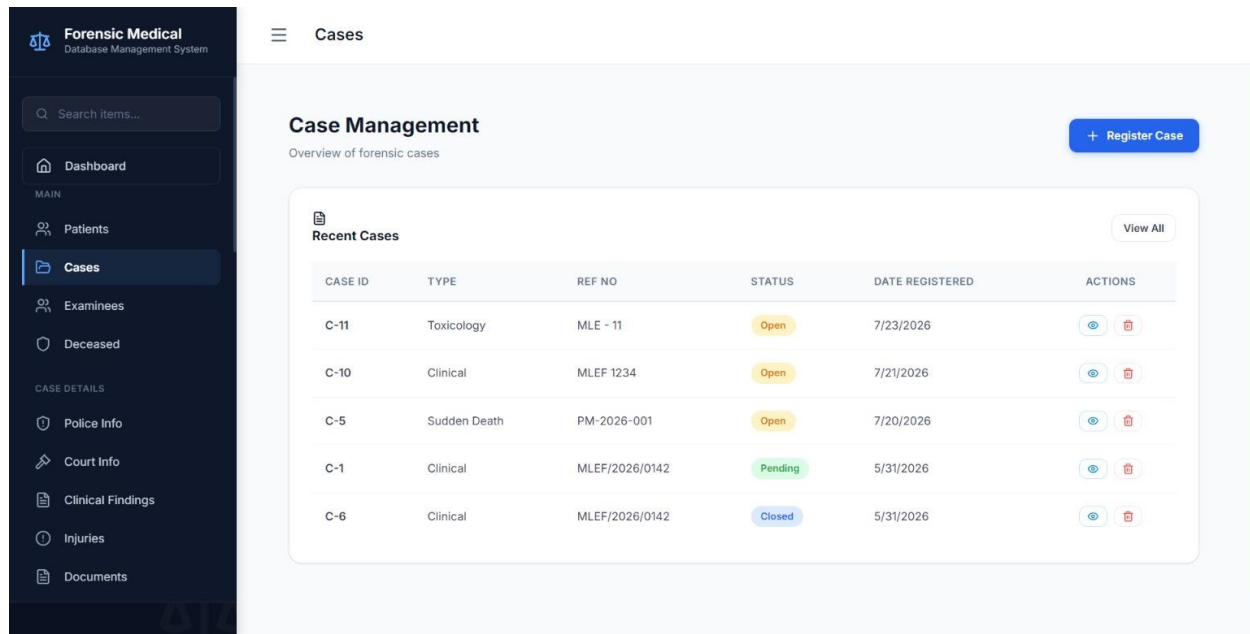
The Patient Management Module allows authorized users to register, update, search, and manage patient records. Patient information such as personal details, NIC, contact information, and medical history are securely stored within the database.

The system validates mandatory fields before saving patient information and prevents duplicate patient registrations using unique constraints.

Functions

- Register Patient
- Edit Patient Information
- Delete Patient
- Search Patient
- View Patient Profile

5.4 Case Management



The Case Management Module manages all forensic investigations. Each registered patient can have one or more forensic cases. Doctors can assign case types, monitor case status, and update investigation progress.

The module links each case to the corresponding patient and assigned doctor through foreign key relationships.

Functions

- Register New Case
- Assign Doctor
- Update Case Status
- Search Case
- View Case Details

5.5 Postmortem Module

The Examination Module stores medical examination findings related to forensic cases. Doctors can record examination types, observations, injuries, and medical conclusions.

Each examination record is linked to the corresponding forensic case.

Functions

- Record Examination

- Update Examination
- View Examination History
- Search Examination Records

The screenshot displays the 'Forensic Medical Database Management System' interface. On the left is a dark sidebar with navigation links: Clinical Findings, Injuries, Documents, INVESTIGATIONS (Investigations, Exhibits, Referrals), FORENSIC & LAB (PM Findings, Specimens, Histopathology, Toxicology), and MEDICAL & REPORTS. The main content area is titled 'PM Findings' and includes a breadcrumb 'Postmortem Findings / All Records'. Below this is a 'Postmortem Findings' section with the subtitle 'Track and manage autopsy results and causes of death' and a '+ New Findings' button. A 'Findings Directory' table is shown with a search bar. The table has columns for FINDING ID, CASE ID, IMMEDIATE CAUSE OF DEATH, MATERNAL DEATH, and ACTIONS. It lists two entries: PMF-2 and PMF-1, both associated with CASE-2 (Closed) and 'Acute myocardial infarction', with 'No' for maternal death. Each entry has view and delete icons in the actions column.

FINDING ID	CASE ID	IMMEDIATE CAUSE OF DEATH	MATERNAL DEATH	ACTIONS
PMF-2	CASE-2 Closed	Acute myocardial infarction	No	
PMF-1	CASE-2 Closed	Acute myocardial infarction	No	

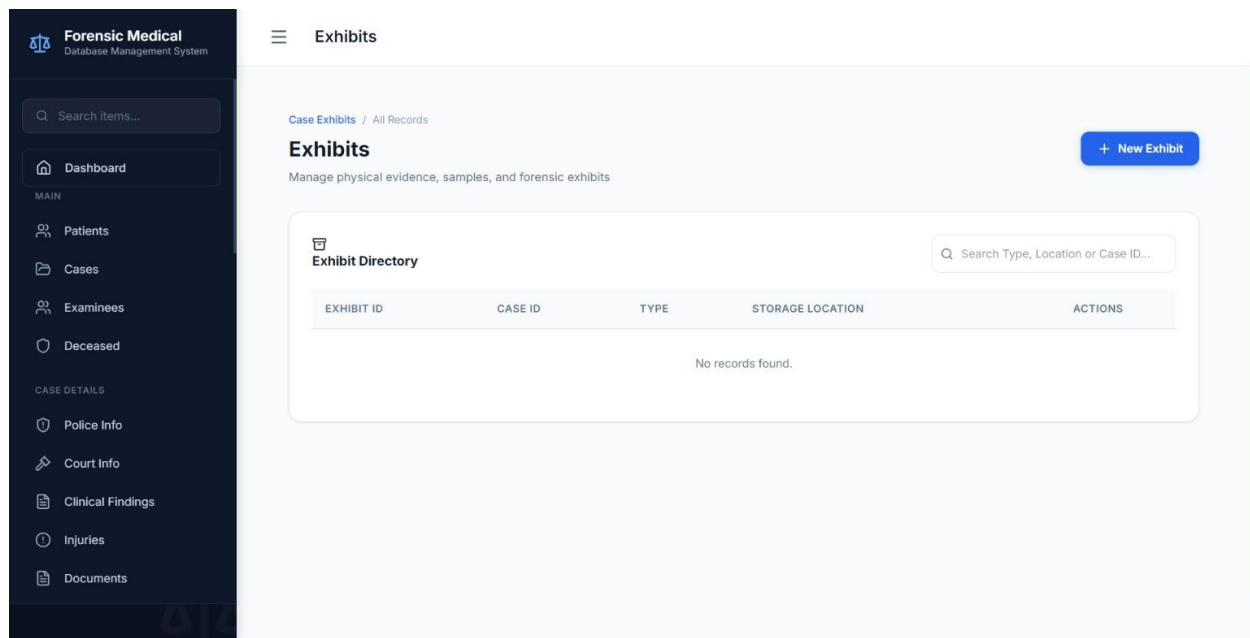
5.6 Evidence Management

This module records and tracks forensic evidence collected during investigations. Each evidence item is associated with a specific forensic case and includes details regarding evidence type, collection date, storage location, and chain of custody.

The module ensures secure management of evidence while preserving its integrity.

Functions

- Add Evidence
- Update Evidence
- Track Evidence
- Search Evidence
- View Chain of Custody

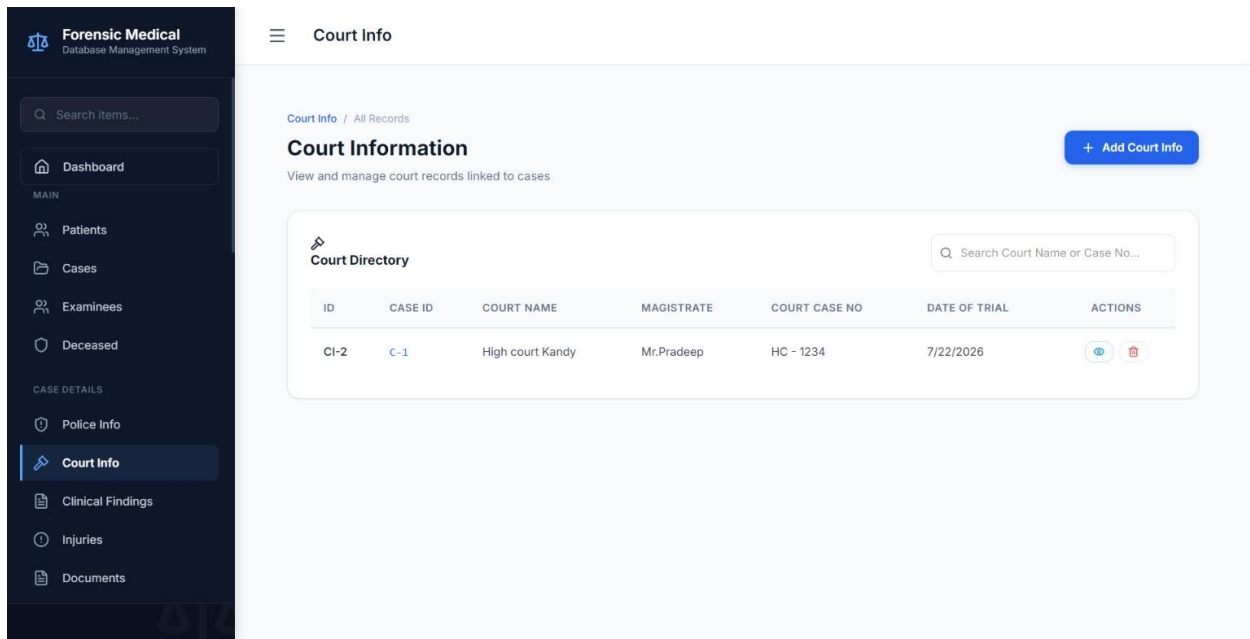


5.7 Court Report Module

The Report Generation Module enables doctors to create Medico-Legal Reports and Postmortem Reports. Reports are generated using data stored within the database and can be viewed, printed, or downloaded.

Functions

- Generate Medico-Legal Report
- Generate Postmortem Report
- Print Report
- Download PDF



5.8 Staff Management

The Staff Management module enables administrators to manage the information of all staff members who use the Forensic Medical Management System. This module maintains employee details such as personal information, job roles, department assignments, contact details, and account status. By centralizing staff information, the system ensures that only authorized personnel have access to sensitive forensic and patient records.

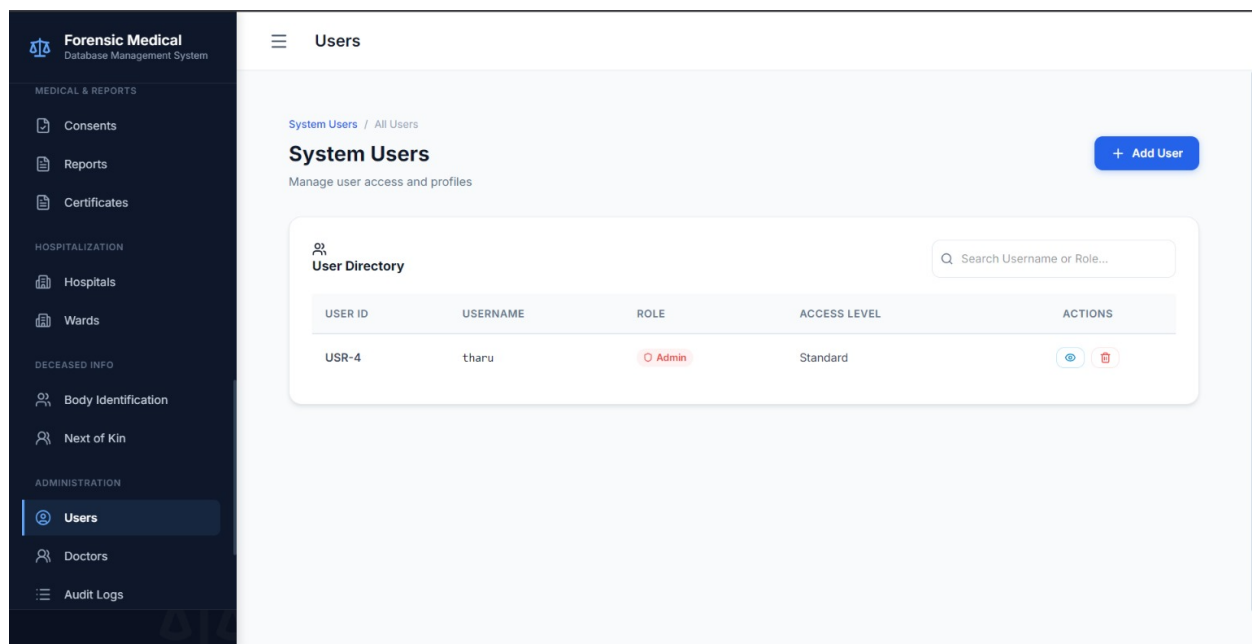
Administrators can register new staff members, update existing employee information, deactivate inactive accounts, and assign appropriate user roles based on job responsibilities. The module also supports searching and filtering staff records, making it easier to locate employee information quickly.

Role-based access control ensures that each staff member can access only the features relevant to their responsibilities. For example, doctors can manage forensic cases and medical reports, while administrative staff can manage patient registrations and departmental information.

Features

- Register new staff members
- Update staff information
- Delete or deactivate staff accounts
- Assign user roles and departments

- Search staff records
- View staff details



5.9 Report Generation

The Report Generation module allows authorized medical officers to generate official forensic documents based on information stored in the database. The system automatically retrieves patient details, forensic case information, examination findings, and evidence records to produce comprehensive reports.

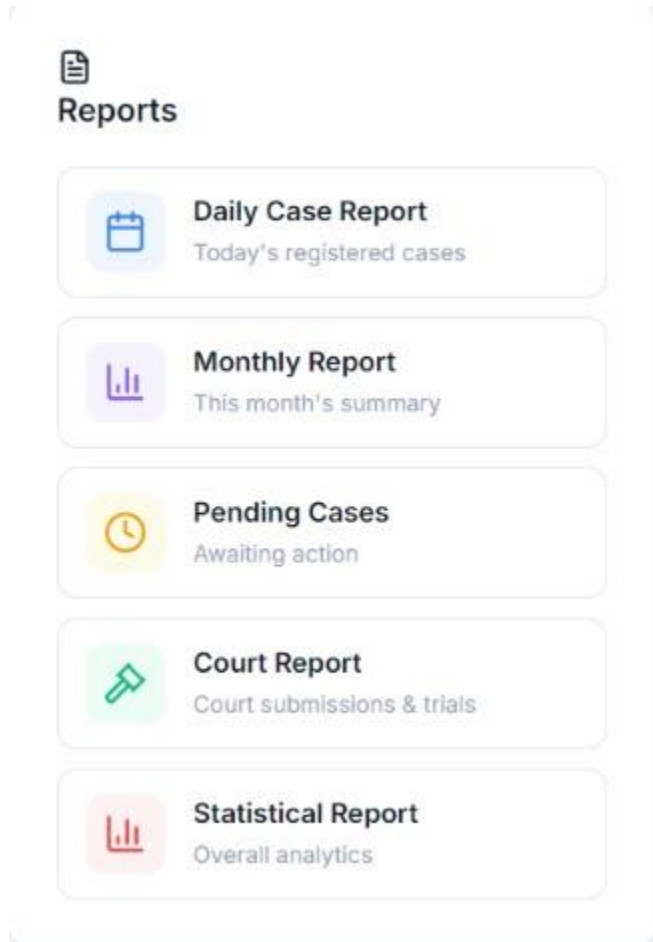
The primary reports generated by the system include Medico-Legal Reports and Postmortem Reports. These reports follow standardized formats and can be viewed, printed, or downloaded for legal and medical purposes.

Automating report generation reduces manual documentation errors, improves efficiency, and ensures consistency across all forensic reports.

Features

- Generate Medico-Legal Reports
- Generate Postmortem Reports

- Preview reports before printing
- Download reports as PDF
- Print reports
- Search previously generated reports



5.10 Dashboard

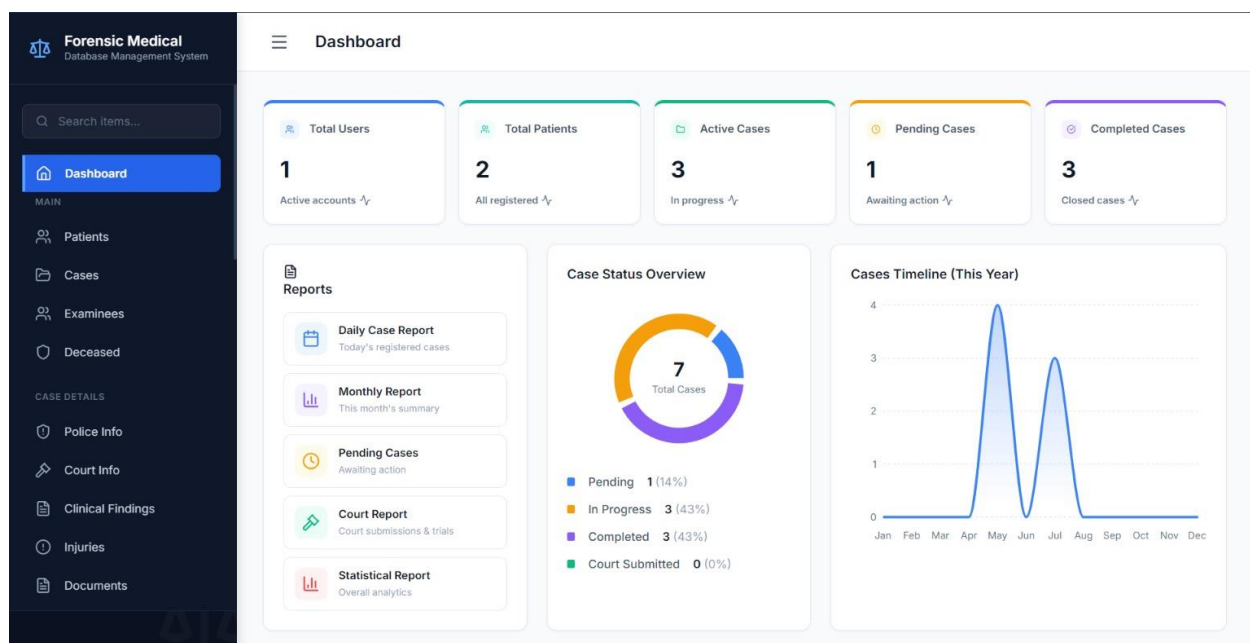
The Dashboard provides users with a centralized overview of the system after successful login. It displays important statistical information, notifications, and shortcuts to frequently used modules. The information displayed varies depending on the user's role within the organization.

For administrators, the dashboard displays summary statistics such as the total number of registered patients, active forensic cases, staff members, completed examinations, pending reports, and evidence records. Doctors can view their assigned cases, scheduled examinations, and recently generated reports. Other users are presented with information relevant to their responsibilities.

Charts, graphs, and summary cards help users quickly understand current system activities and improve decision-making.

Dashboard Components

- Total Registered Patients
- Active Forensic Cases
- Pending Reports
- Completed Examinations
- Registered Staff Members
- Evidence Records
- Recent Activities
- Quick Navigation Menu



5.11 User Authentication

Firebase Authentication is used to securely authenticate all users. Each user must log in using a registered email address and password.

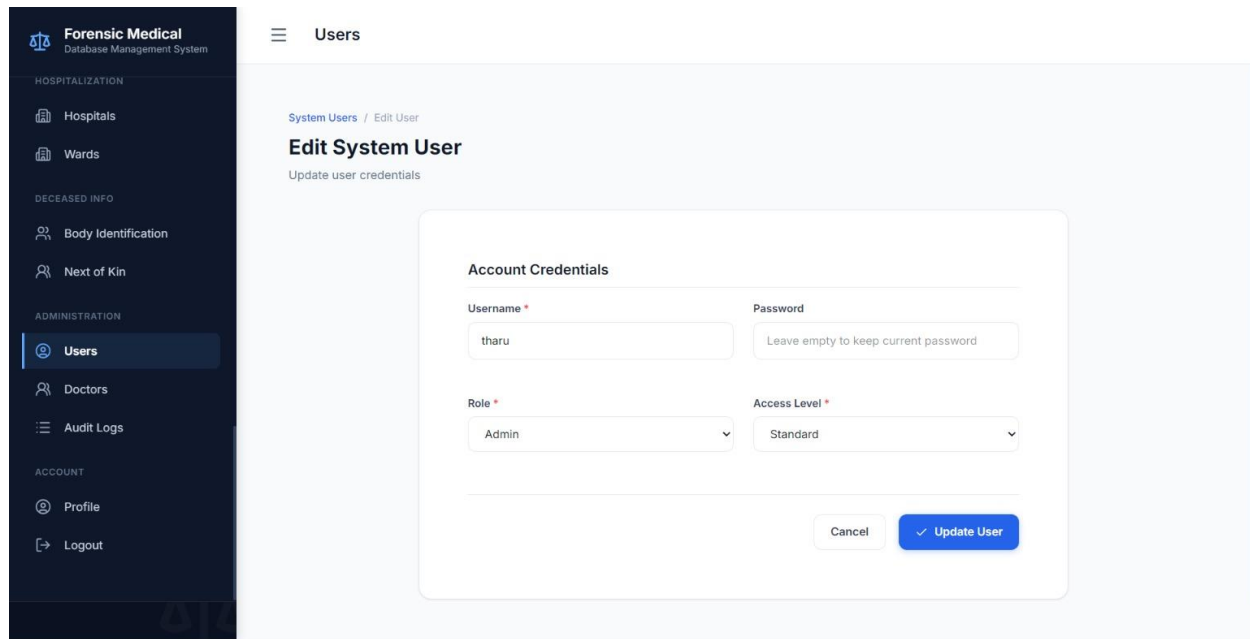
The system uses Role-Based Access Control (RBAC) to ensure users can only access features relevant to their responsibilities.

User Roles

Role	Permissions
Administrator	Full system access
Doctor / JMO	Manage cases, examinations, and reports
Police Officer	Submit police requests and view assigned cases
Laboratory Staff	Manage evidence
Clerical Officer	Register patients
Court Official	View finalized reports

Security Features

- Firebase Authentication
- Secure Login
- Password Encryption
- Session Management
- Role-Based Authorization



Chapter 6 – Testing

6.1 Test Plan

The purpose of the test plan was to verify that every module of the system operates correctly under normal conditions and handles invalid inputs appropriately. Functional testing was performed on the main modules, while SQL query testing was conducted to verify database operations.

The testing process included:

- Login and user authentication testing
- Patient management testing
- Staff management testing
- Forensic case management testing
- Examination management testing
- Evidence management testing
- Report generation testing
- SQL query testing
- Validation testing
- Database backup and recovery testing

Each module was tested using valid and invalid input values to ensure that the system behaved as expected.

6.2 Test Cases

Test Case	Expected Result	Actual Result	Status
User Login	User successfully logs into the system	Login successful	Pass
Invalid Login	Error message displayed	Error displayed	Pass
Register Patient	Patient record saved successfully	Record inserted	Pass
Update Patient	Patient details updated	Updated successfully	Pass
Search Patient	Correct patient information displayed	Correct results shown	Pass
Register Staff	Staff member added successfully	Record inserted	Pass
Create Forensic Case	New case created	Case saved	Pass
Record Examination	Examination saved	Saved successfully	Pass
Register Evidence	Evidence stored successfully	Evidence saved	Pass
Generate Medico-Legal Report	Report generated correctly	Report generated	Pass
Generate Postmortem Report	Report generated correctly	Report generated	Pass
Logout	User session terminated	Logout successful	Pass

6.3 Validation Testing

Validation testing was performed to ensure that invalid or incomplete data could not be stored in the database. Both frontend validation and database constraints were used to maintain data quality.

Examples of validation performed include:

- Mandatory fields cannot be left empty.
- Invalid email addresses are rejected.
- Duplicate NIC numbers are prevented.
- Duplicate doctor registration numbers are not allowed.
- Future dates cannot be selected for completed examinations.
- Invalid user credentials are rejected during login.
- Required foreign key references must exist before records can be inserted.
- Enumeration fields such as gender accept only predefined values.

These validation rules help maintain accurate, consistent, and reliable data within the system.

6.4 SQL Query Testing

6.4.1 SELECT Query

Retrieve all registered patients.

Query 1 x SQL File 3*

Limit to 1000 rows

```

1 • SELECT *
2 FROM Patient;
3

```

Result Grid

	Patient_ID	Name	Gender	Date_of_Birth	Address	Contact_No
*	NULL	NULL	NULL	NULL	NULL	NULL

6.4.2 JOIN Query

Retrieve patient names together with their forensic case details.

```

351 • SELECT
352     c.Case_ID,
353     c.Case_Type,
354     c.Case_Status,
355     d.Name AS Doctor_Name,
356     p.Police_Station
357 FROM `Case` c
358 JOIN Report r
359     ON c.Case_ID = r.Case_ID
360 JOIN Doctor d
361     ON r.Doctor_ID = d.Doctor_ID
362 LEFT JOIN Police_Info p
363     ON c.Case_ID = p.Case_ID;

```

Result Grid

	Case_ID	Case_Type	Case_Status	Doctor_Name	Police_Station
▶	1	Assault	Open	Dr. John Smith	Peradeniya Police Station
	2	RTA	Open	Dr. K. Perera	Gampola Police Station

6.4.3 GROUP BY Query

Count the number of forensic cases assigned to each doctor.

```

365 • SELECT
366     d.Name AS Doctor_Name,
367     COUNT(r.Report_ID) AS Total_Reports
368 FROM Doctor d
369 LEFT JOIN Report r
370     ON d.Doctor_ID = r.Doctor_ID
371 GROUP BY d.Doctor_ID, d.Name;

```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	Doctor_Name	Total_Reports
▶	Dr. John Smith	1
	Dr. K. Perera	1
	Dr. S. Silva	0

6.4.4 Aggregate Function

Calculate the total number of registered patients.

```

372
373 • SELECT COUNT(*) AS Total_Cases
374 FROM `Case`;

```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	Total_Cases
▶	5

6.4.5 Subquery

Retrieve patients who have at least one forensic case.

```

375
376 • SELECT *
377 FROM Doctor
378 WHERE Doctor_ID IN
379 (
380     SELECT Doctor_ID
381     FROM Report
382     WHERE Doctor_ID IS NOT NULL
383 );

```

Result Grid		Filter Rows:		Edit:	Export/Import:		Wrap Cell Content:	
	Doctor_ID	User_ID	Name	Designation	SLMC_Reg_No	Contact_No		
	1	2	Dr. John Smith	JMO	SLMC12345	0771234567		
	2	1	Dr. K. Perera	JMO	SLMC-10234	0711234567		
*	NULL	NULL	NULL	NULL	NULL	NULL		

6.4.6 View

Display all records from the PatientCaseView.

```

385 • SELECT *
386 FROM vw_case_overview;

```

Result Grid		Filter Rows:		Export:	Wrap Cell Content:	
	Case_ID	Case_Type	Case_Status	Date_Registered	Police_Station	Doctor_Name
▶	1	Assault	Open	2026-07-21	Peradeniya Police Station	Dr. John Smith
	2	RTA	Open	2026-07-21	Gampola Police Station	Dr. K. Perera
	3	Sudden Death	Open	2026-07-20	NULL	NULL
	4	Clinical	Open	2026-06-01	NULL	NULL
	5	Autopsy	Closed	2026-05-15	NULL	NULL

6.5 Backup and Recovery

To ensure the availability and reliability of the database, backup and recovery procedures were implemented. Regular database backups protect against accidental data loss, hardware failures, and software errors.

The MySQL database can be backed up using MySQL Workbench or the mysqldump utility. Backup files are stored securely and can be restored whenever necessary.

Backup Procedure

1. Open MySQL Workbench.
2. Navigate to **Server** → **Data Export**.
3. Select the **ForensicMedicalDB** database.
4. Export the database as an SQL dump file.
5. Save the backup in a secure location.

Recovery Procedure

1. Open MySQL Workbench.
2. Navigate to **Server** → **Data Import**.
3. Select the SQL backup file.
4. Choose the target database.
5. Execute the import to restore all tables and data.

Chapter 7 – Reports

7.1 Daily Case Report

The Daily Case Report provides a summary of all forensic cases registered on a specific date. It includes essential information such as case identification numbers, patient names, assigned doctors, case types, and current case statuses. This report assists administrators and medical officers in monitoring daily workload and tracking newly registered cases.

Information Included

- Case ID
- Patient Name
- Doctor Name
- Case Type
- Date Registered
- Current Status

Purpose

- Monitor daily forensic activities
- Track newly registered cases
- Assist daily administrative planning
- Improve case monitoring

Forensic Medicine Reporting System
Daily Case Report
Generated on: 2026-07-23

CASE ID	MLEF/PM NO	TYPE	STATUS	PERSON	DOCTOR	POLICE STATION	IO
11	MLE - 11	Toxicology	Open	N/A	Unassigned	-	-

Authorized Signature

Date

7.2 Monthly Report

The Monthly Report summarizes forensic activities performed during a selected month. It provides statistical information regarding the number of registered patients, completed examinations, evidence records, medico-legal reports, and postmortem reports.

This report enables management to evaluate departmental performance and identify workload trends over time.

Information Included

- Total Patients Registered
- Total Forensic Cases
- Total Examinations
- Total Evidence Records
- Total Medico-Legal Reports
- Total Postmortem Reports

Purpose

- Monthly performance evaluation
- Workload analysis
- Departmental reporting
- Resource planning

Forensic Medicine Reporting System						
Monthly Report						
Generated on: July 2026						
Open 3						
CASE ID	MLEF/PM NO	TYPE	STATUS	DATE REGISTERED	PERSON	DOCTOR
11	MLE - 11	Toxicology	Open	23/07/2026	N/A	Unassigned
10	MLEF 1234	Clinical	Open	21/07/2026	N/A	Unassigned
5	PM-2026-001	Sudden Death	Open	20/07/2026	N/A	Unassigned
Authorized Signature			Date			

7.3 Pending Cases Report

The Pending Cases Report displays all forensic cases that have not yet been completed. Cases are filtered according to their current status, allowing authorized users to identify ongoing investigations requiring further attention.

This report assists doctors and administrators in prioritizing unfinished cases and monitoring investigation progress.

Information Included

- Case ID
- Patient Name
- Assigned Doctor
- Case Type
- Registration Date

- Current Status

Purpose

- Monitor ongoing investigations
- Reduce reporting delays
- Improve case management
- Track incomplete examinations

Forensic Medicine Reporting System							
Pending Cases Report							
Generated on: 2026-07-23							
CASE ID	MLEF/PM NO	TYPE	DATE REGISTERED	DAYS PENDING	PERSON	DOCTOR	POLICE
1	MLEF/2026/0142	Clinical	31/05/2026	53	A. Fernando	Unassigned	Perad Police
Authorized Signature				Date			

7.4 Court Report

The Court Report contains finalized medico-legal information required by judicial authorities. It summarizes patient information, forensic examination findings, evidence details, and medical conclusions prepared by the responsible medical officer.

Only authorized users such as doctors and court officials are permitted to access this report.

Information Included

- Report Number
- Case Number
- Patient Details
- Examination Findings
- Medical Opinion
- Evidence Summary
- Doctor Details

- Report Date

Purpose

- Support legal proceedings
- Provide official medico-legal documentation
- Improve information accuracy
- Facilitate court presentations

Forensic Medicine Reporting System

Court Report

Generated on: 2026-07-23

CASE ID	MLEF/PM NO	COURT	MAGISTRATE	COURT CASE NO	TRIAL DATE	PERSON	DOCTOR
1	MLEF/2026/0142	High court Kandy	Mr.Pradeep	HC - 1234	22/07/2026	A. Fernando	Unassign

◀ ▶

Authorized Signature

Date

Court Report

7.5 Statistical Report

The Statistical Report presents graphical and numerical summaries of data stored in the database. It assists administrators in analysing trends related to forensic activities and departmental performance.

Charts and summary statistics provide valuable insights into patient registrations, case distributions, report generation, and evidence management.

Information Included

- Total Patients
- Total Staff Members

- Active Cases
- Completed Cases
- Pending Cases
- Total Evidence Records
- Monthly Case Distribution
- Report Generation Statistics

Purpose

- Performance monitoring
- Trend analysis
- Decision support
- Department planning

Forensic Medicine Reporting System

Statistical Report

Generated on: 2026-07-23

CASES

7

PATIENTS

2

DOCTORS

5

REPORTS

1

Cases by Type

CASE TYPE	COUNT
Clinical	3
Autopsy	2
Sudden Death	1
Toxicology	1

Cases by Status

STATUS	COUNT
Closed	3
Open	3
Pending	1

Statistical Report Dashboard

Chapter 8 – Security

8.1 Introduction

Security is one of the most critical aspects of the Forensic Medical Management System because it stores highly confidential patient records, forensic case information, examination findings, evidence details, and medico-legal reports. Unauthorized access or modification of this information could compromise legal investigations and patient privacy.

To ensure the confidentiality, integrity, and availability of data, several security mechanisms were implemented within the system. These mechanisms include secure user authentication, role-based access control, password protection, audit logging, database backups, and controlled access to sensitive information.

8.2 Password Security

The system uses **Firestore Authentication** to provide secure user authentication. Users are required to log in using their registered email address and password before accessing any protected resources.

Passwords are never stored directly in the application's database. Instead, Firestore securely manages user credentials using industry-standard encryption and authentication mechanisms. This approach minimizes the risk of password theft and protects user accounts from unauthorized access.

Additional password security measures include:

- Secure email and password authentication.
- Password reset functionality via email.
- Encrypted password storage handled by Firestore.
- Automatic rejection of invalid login credentials.
- Session termination after logout.

These measures ensure that user credentials remain secure throughout the authentication process.

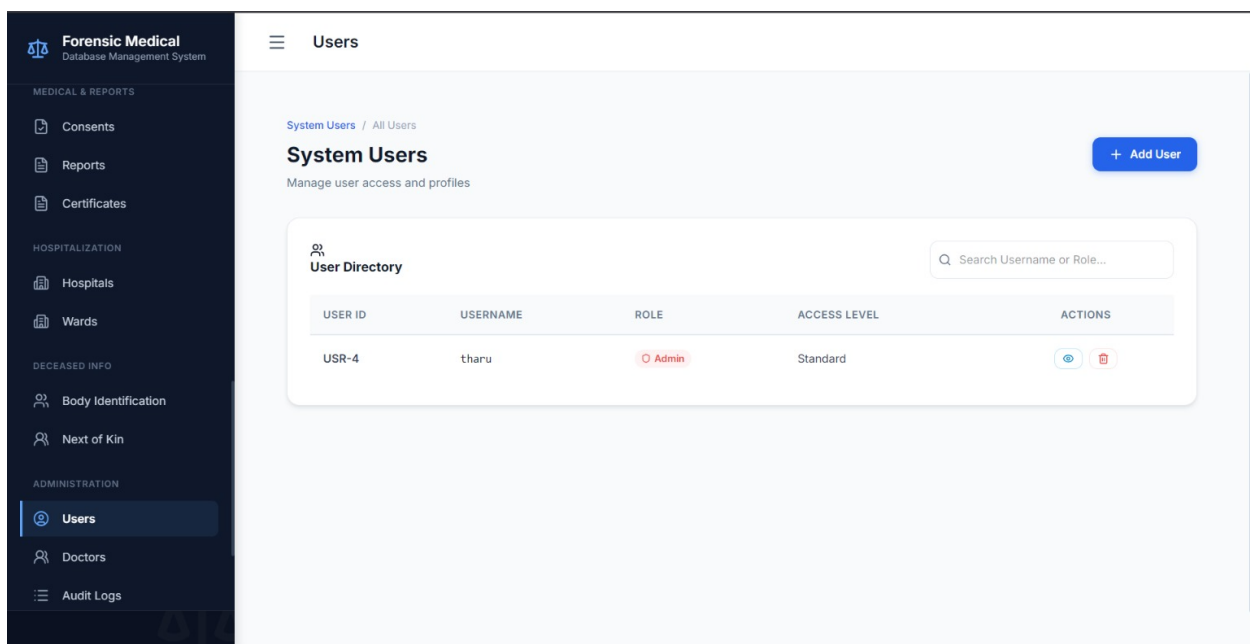
8.3 User Roles

The system implements **Role-Based Access Control (RBAC)**, where users are assigned specific roles based on their responsibilities within the forensic medical department. Each role is granted only the permissions necessary to perform its assigned tasks.

The main user roles are shown below.

User Role	Responsibilities
Administrator	Manage users, staff, departments, and system settings
Doctor / Judicial Medical Officer	Manage forensic cases, examinations, and medico-legal reports
Police Officer	Submit police requests and monitor assigned cases
Laboratory Staff	Register and manage forensic evidence
Clerical Officer	Register patients and maintain patient records
Court Official	View finalized medico-legal reports for legal proceedings

This role-based approach minimizes unauthorized access and protects sensitive forensic information.



User Role Management Page

8.4 Access Control

Access control ensures that users can access only the resources permitted by their assigned roles. After successful authentication, the system verifies the user's role before granting access to protected modules.

For example:

- Only administrators can manage staff accounts.
- Doctors can create examinations and generate medico-legal reports.

- Laboratory staff can register and update evidence.
- Police officers can submit police requests but cannot modify medical reports.
- Court officials have read-only access to finalized reports.

Unauthorized users attempting to access restricted pages are redirected or shown an appropriate access denied message.

This mechanism prevents unauthorized modifications and helps maintain the integrity of forensic records.

8.5 Confidentiality

Maintaining confidentiality is essential because the system stores sensitive personal and medico-legal information.

The following measures help protect confidential data:

- Access is restricted to authenticated users only.
- Sensitive information is available only to authorized personnel.
- Patient information is stored in a centralized relational database.
- Role-based permissions prevent unauthorized viewing or modification of records.
- Database transactions maintain data consistency and integrity.
- Secure communication between the frontend and backend helps prevent unauthorized interception of data.

These mechanisms ensure compliance with organizational policies and improve trust in the forensic medical record management process.

8.6 Audit Logs

Audit logging provides accountability by recording important user activities performed within the system. Audit logs help administrators monitor system usage and investigate security incidents if unauthorized actions occur.

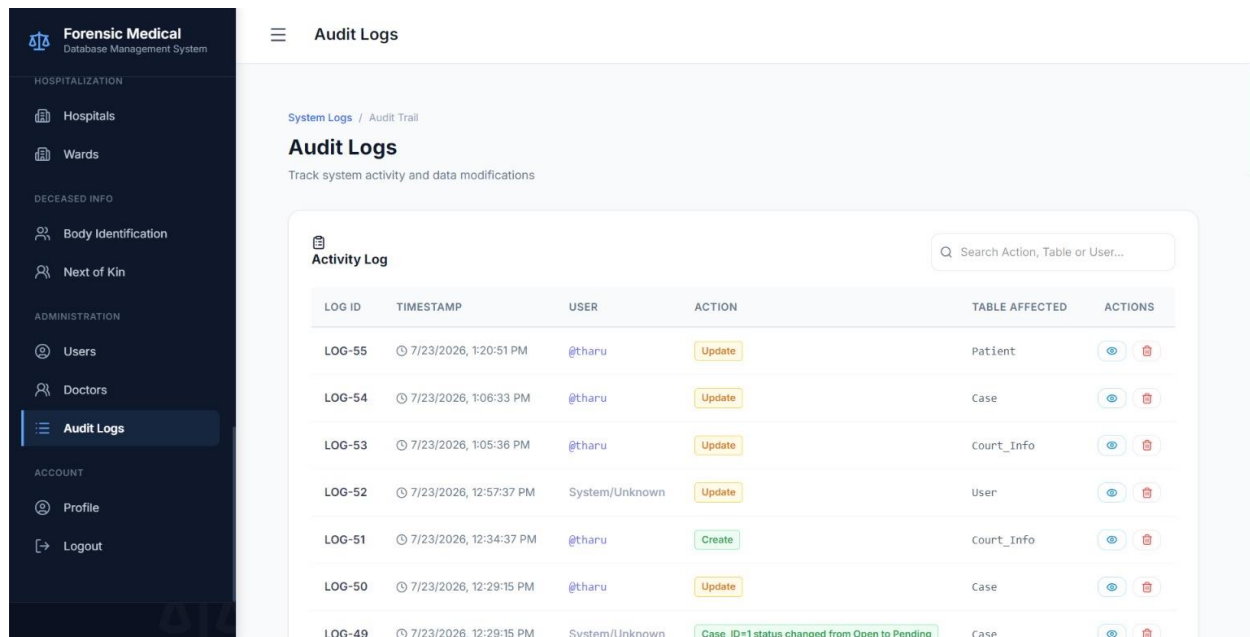
Examples of activities recorded include:

- User login and logout
- Patient registration
- Case creation
- Evidence updates
- Report generation
- Staff management activities

Each audit record may include:

- User ID
- User role
- Activity performed
- Date and time
- Affected module

Audit logs improve system transparency and support forensic investigations by maintaining a history of important operations.



LOG ID	TIMESTAMP	USER	ACTION	TABLE AFFECTED	ACTIONS
LOG-55	7/23/2026, 1:20:51 PM	@tharu	Update	Patient	 
LOG-54	7/23/2026, 1:06:33 PM	@tharu	Update	Case	 
LOG-53	7/23/2026, 1:05:36 PM	@tharu	Update	Court_Info	 
LOG-52	7/23/2026, 12:57:37 PM	System/Unknown	Update	User	 
LOG-51	7/23/2026, 12:34:37 PM	@tharu	Create	Court_Info	 
LOG-50	7/23/2026, 12:29:15 PM	@tharu	Update	Case	 
LOG-49	7/23/2026, 12:29:15 PM	System/Unknown	Case_ID=1 status changed from Open to Pending	Case	 

Audit Log Page (or Activity Log Table)

8.7 Database Backup

Regular database backups are essential to ensure data availability and recoverability in the event of accidental deletion, hardware failure, or software errors.

The MySQL database can be backed up using MySQL Workbench or the mysqldump utility. Backup files are stored securely and can be restored whenever required.

Example backup command:

```
mysqldump -u root -p ForensicMedicalDB > ForensicMedicalDB_Backup.sql
```

Example restore command:

```
mysql -u root -p ForensicMedicalDB < ForensicMedicalDB_Backup.sql
```

Regular backups ensure that patient records, forensic cases, evidence information, and medico-legal reports can be recovered with minimal data loss.

Backup Procedure

1. Open MySQL Workbench.
2. Navigate to **Server** → **Data Export**.
3. Select the **ForensicMedicalDB** database.
4. Export the database as an SQL dump file.
5. Store the backup in a secure location.

Recovery Procedure

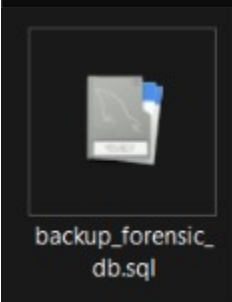
1. Open MySQL Workbench.
2. Navigate to **Server** → **Data Import**.
3. Select the backup SQL file.
4. Choose the destination database.
5. Restore all tables and records.

```
Microsoft Windows [Version 10.0.26300.8935]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Tharusha>mysql_config_editor set --login-path=local --host=localhost --user=root --password
Enter password: *****

C:\Users\Tharusha>mysqldump --login-path=local forensic_medicine_db > "D:\backup_forensic_db.sql"

C:\Users\Tharusha>
```



Chapter 9 – Challenges and Future Improvements

9.1 Introduction

The development of the Forensic Medical Management System involved designing a secure and reliable database capable of managing sensitive medico-legal information. Although the project successfully achieved its primary objectives, several challenges were encountered during the analysis, design, implementation, and testing phases. These challenges provided valuable learning experiences and highlighted areas where the system can be further enhanced.

Future improvements are proposed to extend the system's functionality, improve user experience, strengthen security, and support emerging technologies.

9.2 Challenges Faced During Development

9.2.1 Requirement Gathering

One of the major challenges was understanding the workflow of a forensic medical department. The system needed to accommodate multiple stakeholders, including doctors, police officers, laboratory staff, clerical officers, administrators, and court officials, each with different responsibilities and data requirements.

To address this challenge, the project team analyzed the forensic workflow, identified key stakeholders, and translated their responsibilities into functional requirements.

9.2.2 Database Design Complexity

Designing the database required careful identification of entities, attributes, and relationships. The system manages multiple interconnected entities such as patients, forensic cases, police

requests, examinations, evidence records, medico-legal reports, postmortem reports, doctors, and departments.

Developing an efficient relational schema while minimizing redundancy required normalization up to Third Normal Form (3NF). Primary keys, foreign keys, and integrity constraints were carefully implemented to maintain consistency and prevent invalid relationships.

9.2.3 Maintaining Data Integrity

Ensuring data integrity was another significant challenge. Since many tables depend on one another through foreign key relationships, inserting or deleting records in an incorrect order could result in constraint violations.

To overcome this issue, primary keys, foreign keys, unique constraints, and validation rules were implemented to ensure that only valid data are stored in the database.

9.2.4 User Interface Design

The system required a user-friendly interface suitable for users with different technical backgrounds. Designing forms that were simple, responsive, and easy to navigate while supporting complex forensic workflows required careful planning.

React.js and Bootstrap were used to develop a responsive and intuitive user interface that simplifies patient registration, case management, evidence recording, and report generation.

9.2.5 Authentication and Security

Protecting sensitive medico-legal information was a major priority. Implementing secure authentication and restricting access to confidential records required integration with Firebase Authentication and role-based access control.

Proper authentication mechanisms ensure that only authorized users can access restricted modules and perform operations based on their assigned roles.

9.2.6 SQL Query Optimization

Generating reports involving multiple related tables required writing efficient SQL queries. Complex JOIN operations and aggregate queries initially resulted in slower execution when working with larger datasets.

Indexes were added to frequently searched attributes and SQL queries were optimized to improve retrieval performance.

9.2.7 Testing and Debugging

Testing involved verifying the correctness of frontend components, backend APIs, and SQL queries. Several issues were identified during testing, including validation errors, foreign key constraint violations, and incorrect query results.

These issues were resolved through systematic debugging and repeated testing until the expected functionality was achieved.

9.3 Future Improvements

Although the current system successfully manages forensic medical records, several enhancements can further improve its functionality and usability.

9.3.1 QR Code Integration

QR codes can be generated for each forensic case to enable quick identification and retrieval of case information. Authorized personnel could scan the QR code to access relevant case details securely.

Benefits

- Faster case identification
- Reduced manual searching
- Improved evidence tracking

9.3.2 Mobile Application

Developing Android and iOS applications would allow doctors and authorized staff to access the system remotely using smartphones or tablets.

Benefits

- Mobile access
- Improved field operations
- Faster report submission

9.3.3 SMS and Email Notifications

Automatic notifications could inform users when important events occur, such as:

- New case registration
- Examination scheduling
- Report completion
- Court report availability

Benefits

- Improved communication
- Reduced delays
- Better workflow management

9.3.4 AI-Assisted Report Generation

Artificial Intelligence could assist doctors by generating preliminary medico-legal reports based on examination findings entered into the system.

AI could also help identify missing information, suggest medical terminology, and improve report consistency.

Benefits

- Reduced documentation time
- Improved report quality
- Decision support

9.3.5 Cloud Deployment

Deploying the system on cloud platforms such as AWS, Microsoft Azure, or Google Cloud Platform would improve scalability, reliability, and accessibility.

Benefits

- High availability
- Automatic backups
- Easier maintenance
- Remote access

9.3.6 Digital Signatures

Digital signatures could be incorporated into medico-legal and postmortem reports to verify authenticity and prevent unauthorized modifications.

Benefits

- Improved document security
- Legal authenticity
- Reduced document forgery

9.3.7 Biometric Authentication

Fingerprint or facial recognition authentication could provide an additional layer of security for doctors and administrators accessing sensitive medico-legal records.

Benefits

- Stronger authentication
- Reduced password-related risks
- Improved system security

9.3.8 Advanced Analytics Dashboard

Future versions of the system could include interactive dashboards that display trends in forensic cases, evidence management, report generation, and departmental performance using charts and graphs.

Benefits

- Better decision-making
- Trend analysis
- Performance monitoring

9.3.9 Integration with Hospital and Police Systems

The system could be integrated with hospital information systems and police databases to automatically exchange patient referrals, case details, and investigation updates.

Benefits

- Reduced duplicate data entry
- Faster information sharing
- Improved collaboration

Chapter 10 – Conclusion

10.1 Introduction

The **Forensic Medical Management System** was developed to provide a centralized, secure, and efficient platform for managing medico-legal information within a forensic medical department. The project addressed the limitations of traditional paper-based record management by implementing a modern database-driven solution capable of handling patient information, forensic cases, medical examinations, evidence records, police requests, and report generation.

The system was designed using sound database design principles, including Entity-Relationship (ER) modeling, relational schema design, normalization, and integrity constraints. A web-based application was developed using React.js, Node.js, Express.js, MySQL, and Firebase Authentication to provide a secure and user-friendly interface for authorized users.

10.2 Objectives Achieved

The primary objective of this project was to design and implement a complete database management system for a forensic medical department. The developed system successfully achieved the objectives identified during the requirement analysis.

The major objectives accomplished include:

- Designed a normalized relational database up to Third Normal Form (3NF) to reduce redundancy and improve data consistency.
- Developed a centralized database to manage patients, forensic cases, examinations, evidence records, police requests, and reports.
- Implemented primary keys, foreign keys, and integrity constraints to maintain accurate relationships among database tables.
- Developed a responsive web application for efficient data entry and retrieval.
- Implemented Firebase Authentication to provide secure user login and role-based access control.

- Enabled efficient searching, updating, and management of forensic medical records.
- Automated the generation of medico-legal and postmortem reports.
- Implemented SQL queries, views, indexes, stored procedures, and triggers to improve database functionality and performance.
- Performed testing to verify the correctness and reliability of both the application and the database.

The successful completion of these objectives demonstrates that the proposed system effectively supports the management of forensic medical information.

10.3 Benefits of the System

The developed system provides several benefits to forensic medical departments by improving the efficiency, accuracy, and security of medico-legal record management.

Improved Record Management

The centralized database allows all patient, case, examination, evidence, and report information to be stored in a single location, reducing duplication and simplifying information retrieval.

Enhanced Data Integrity

The use of relational database concepts such as primary keys, foreign keys, unique constraints, and validation rules ensures that data remain accurate, consistent, and reliable.

Improved Security

Firestore Authentication and Role-Based Access Control restrict access to sensitive information, ensuring that only authorized users can perform specific operations within the system.

Efficient Report Generation

The automated report generation module reduces manual documentation effort and enables doctors to produce standardized medico-legal and postmortem reports more efficiently.

Better Decision Making

Search functionality, dashboards, and statistical reports provide administrators with timely information that supports monitoring, planning, and decision-making.

Reduced Paperwork

Digitizing forensic records minimizes dependence on paper documents, reducing storage requirements and the risk of document loss or damage.

Scalability

The modular architecture and normalized database design provide a strong foundation for future expansion, including additional modules and integration with external systems.

10.4 Lessons Learned

The development of this project provided valuable practical experience in both database design and full-stack web application development.

The project strengthened the team's understanding of:

- Requirement analysis and system planning.
- Entity-Relationship modeling.
- Relational database design and normalization.
- SQL programming and database implementation.
- Primary keys, foreign keys, constraints, indexes, views, stored procedures, and triggers.
- React.js frontend development.
- Backend API development using Node.js and Express.js.
- Firebase Authentication and user authorization.
- Software testing and debugging.
- Team collaboration and project management.

The project also demonstrated the importance of careful database design in developing secure, reliable, and scalable information systems.

10.5 Overall Conclusion

The **Forensic Medical Management System** successfully addresses many of the limitations associated with traditional manual record-keeping methods. By providing a centralized relational database and an intuitive web-based interface, the system improves the management of forensic cases, patient records, evidence, examinations, and medico-legal reports.

The integration of modern technologies such as React.js, Node.js, Express.js, MySQL, and Firebase Authentication has resulted in a secure, responsive, and efficient application that supports the daily operations of a forensic medical department. Database normalization, integrity constraints, and structured SQL implementation ensure reliable data storage while minimizing redundancy and maintaining consistency.

Although there are opportunities for future enhancements, such as mobile applications, cloud deployment, AI-assisted reporting, and biometric authentication, the current implementation successfully fulfills the objectives defined at the beginning of the project. It demonstrates the practical application of database management concepts while providing a functional solution for managing forensic medical information.

10.6 Final Remarks

The project demonstrates how database technologies can significantly improve the management of sensitive medico-legal information. By replacing fragmented and paper-based processes with a secure, centralized digital system, the Forensic Medical Management System enhances operational efficiency, data accuracy, and information security.

Furthermore, this project provided valuable practical experience in database systems, software engineering, and web application development. The knowledge gained throughout the project will be beneficial for the design and implementation of future information systems in healthcare, forensic medicine, and other data-intensive domains.

References

- [1] Oracle Corporation, *MySQL 8.0 Reference Manual*. [Online]. Available: <https://dev.mysql.com/doc/>
- [2] Oracle Corporation, *MySQL Workbench User Guide*. [Online]. Available: <https://dev.mysql.com/doc/workbench/en/>
- [3] Firebase, *Firebase Authentication Documentation*. [Online]. Available: <https://firebase.google.com/docs/auth>
- [4] React Team, *React Documentation*. [Online]. Available: <https://react.dev/>
- [5] Node.js Foundation, *Node.js Documentation*. [Online]. Available: <https://nodejs.org/docs>
- [6] Express.js, *Express Web Framework*. [Online]. Available: <https://expressjs.com/>
- [7] Mozilla Developer Network (MDN), *JavaScript Guide*. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [8] Elmasri, R., & Navathe, S. B., *Fundamentals of Database Systems*, 7th Edition, Pearson, 2017.
- [9] Silberschatz, A., Korth, H. F., & Sudarshan, S., *Database System Concepts*, 7th Edition, McGraw-Hill, 2020.
- [10] W3Schools, *SQL Tutorial*. [Online]. Available: <https://www.w3schools.com/sql/>
- [11] GeeksforGeeks, *DBMS Tutorials*. [Online]. Available: <https://www.geeksforgeeks.org/dbms/>

[12] Bootstrap Team, *Bootstrap Documentation*. [Online]. Available: <https://getbootstrap.com/docs/>

Individual Contribution

Student (Name and Eno)	Role	Tasks Completed	Contribution (%)
E/22/180 - K.A.D Geethma Chandooa	Team Leader	Documentation	20%
E/22/132 - R.M.Y Hansira	Web Interface Developer	UI /UX designing and developing	30%
E/22/154 - S.A.T Inuwara	Database Designer	Connecting SQL fles with the backend	30%
E/22/288 - S.L.Y Prabudika	Requirement Analyst,Testing	Recognizing Functional and non functional requirements	20%

Each student should explain:

- Responsibilities
- Tasks completed
- Challenges faced
- Lessons learned

Project Timeline

Week	Activity	Status
1	Requirement Analysis	completed

Week	Activity	Status
2	Database Design	completed
3	Interface Design	completed
4	Database Implementation	completed
5	Testing & Reports	completed
6	Documentation & Presentation	completed